

OPMS Software Documentation

Complete documentation suite (Markdown → PDF with Mermaid diagrams)

Generated: 2026-09-22

Source: repository docs/ directory

Diagrams rendered with Mermaid 11

Table of Contents

- README.md
- DOCUMENTATION-AUDIT.md
- 01-product/01-product-overview.md
- 01-product/02-prd.md
- 01-product/03-scope.md
- 01-product/04-personas-and-roles.md
- 01-product/05-features.md
- 01-product/06-business-requirements.md
- 01-product/07-non-functional-requirements.md
- 01-product/08-business-rules.md
- 01-product/09-integrations.md
- 01-product/10-product-glossary.md
- 02-architecture/01-system-architecture.md
- 02-architecture/02-component-architecture.md
- 02-architecture/03-frontend-architecture.md
- 02-architecture/04-backend-architecture.md
- 02-architecture/05-service-architecture.md
- 02-architecture/06-data-flow.md
- 02-architecture/07-authentication-authorization.md
- 02-architecture/08-integration-architecture.md
- 02-architecture/09-infrastructure-architecture.md
- 02-architecture/10-deployment-architecture.md
- 02-architecture/11-architecture-decisions.md
- 02-architecture/12-adr/ADR-001-microservice-split.md
- 02-architecture/12-adr/ADR-002-opms-backend-bff.md
- 02-architecture/12-adr/ADR-003-shared-mongodb.md
- 02-architecture/12-adr/ADR-004-jwt-portal-rbac.md
- 02-architecture/12-adr/ADR-005-bullmq-redis.md
- 02-architecture/12-adr/ADR-006-micro-frontends-sso.md
- 02-architecture/12-adr/ADR-007-external-file-api.md
- 02-architecture/12-adr/ADR-008-unified-order-approval.md
- 03-functional/README.md
- 03-functional/authentication.md
- 03-functional/users-and-identity.md
- 03-functional/orders-and-workflow.md
- 03-functional/approvals-and-finance.md
- 03-functional/dispatch-and-transport.md
- 03-functional/parties-and-products.md
- 03-functional/leads-and-quotations.md
- 03-functional/work-planner.md

03-functional/notifications-and-messages.md
03-functional/attachments-and-files.md
03-functional/dashboards-and-activity.md
04-api/01-api-overview.md
04-api/02-authentication.md
04-api/03-api-conventions.md
04-api/04-error-handling.md
04-api/05-endpoints.md
05-database/01-database-overview.md
05-database/02-schema.md
05-database/03-collections-or-tables.md
05-database/04-relationships.md
05-database/05-indexes.md
05-database/06-constraints.md
05-database/07-data-lifecycle.md
05-database/08-data-dictionary.md
05-database/09-erd.md
06-developer-deployment/01-development-overview.md
06-developer-deployment/02-prerequisites.md
06-developer-deployment/03-local-development.md
06-developer-deployment/04-project-structure.md
06-developer-deployment/05-environment-variables.md
06-developer-deployment/06-configuration.md
06-developer-deployment/07-running-the-application.md
06-developer-deployment/08-testing.md
06-developer-deployment/09-code-quality.md
06-developer-deployment/10-git-workflow.md
06-developer-deployment/11-docker.md
06-developer-deployment/12-deployment.md
06-developer-deployment/13-production-runbook.md
06-developer-deployment/14-monitoring.md
06-developer-deployment/15-logging.md
06-developer-deployment/16-backup-and-restore.md
06-developer-deployment/17-disaster-recovery.md
06-developer-deployment/18-troubleshooting.md
06-developer-deployment/19-security.md
06-developer-deployment/20-performance.md
07-user/01-user-guide.md
07-user/02-getting-started.md
07-user/03-login.md
07-user/04-navigation.md
07-user/05-feature-guides.md
07-user/06-role-guides.md
07-user/07-common-tasks.md

07-user/08-troubleshooting.md

07-user/09-faq.md

OPMS Software Documentation

Product: OPMS (Order / Operations / Portal Management System) — multi-portal business platform for order lifecycle, CRM (leads), field work planning, identity/user management, and communications.

Repository: Monorepo at repository root (`opms 2`) containing 8 Node.js backend services, 5 Next.js frontends, Docker Compose, and host nginx TLS configs.

Documentation status: Generated from the current codebase. Where the repository does not define something, documents mark it as **Not identified**, **Unknown**, or **Requires confirmation**.

Important: Do not treat `opms-backend/docs/essentials/*` as the live workflow source of truth. Live order transitions are in `opms-backend/src/modules/workflow/workflow.transitions.js`.

1. Product

Document	Description
Product Overview	Purpose, users, capabilities, boundaries
PRD	Vision, user stories, FR/NFR, acceptance criteria
Scope	In-scope / out-of-scope
Personas and Roles	Portals and access roles
Features	Feature catalog by portal
Business Requirements	Business requirements traced to code
Non-Functional Requirements	Security, ops, maintainability
Business Rules	Workflow, RBAC, blocking rules
Integrations	External systems
Product Glossary	Domain terms

2. Architecture

Document	Description
System Architecture	Context and high-level topology
Component Architecture	Services and responsibilities
Frontend Architecture	Micro-frontends and SSO hub
Backend Architecture	Express services, BFF pattern
Service Architecture	Service-to-service map
Data Flow	Key data flows
Authentication & Authorization	JWT, portals, roles
Integration Architecture	Email, WhatsApp, files, sheets
Infrastructure Architecture	Docker, Redis, Mongo, nginx

Document	Description
Deployment Architecture	Deploy topology
Architecture Decisions	ADR index
ADRs	Individual decision records

3. Functional

Document	Description
Authentication	Login, SSO, session
Users & Identity	Users, departments, portals
Orders & Workflow	Order lifecycle
Approvals & Finance	Approvals, due sheets, unbilled
Dispatch & Transport	Dispatch, fleet, transport planner
Parties & Products	Masters and rates
Leads & Quotations	Lead Manager CRM
Work Planner	Plans, visits, expenses
Notifications & Messages	Push, email, WhatsApp
Attachments & Files	Uploads and file API
Dashboards & Activity	KPI dashboards

4. API

Document	Description
API Overview	Gateways, ports, base paths
Authentication	JWT and portal gates
API Conventions	Patterns, soft-delete
Error Handling	Status codes and errors
Endpoints	Full endpoint inventory
OpenAPI	Generated OpenAPI 3.0 inventory

Live Swagger UI (opms-backend only): `GET /api-docs`

Source: `opms-backend/src/docs/swagger.js` , `swaggerExtended.js`

5. Database

Document	Description
Database Overview	MongoDB usage
Schema	Schema ownership
Collections	Collection catalog
Relationships	Logical relationships

Document	Description
Indexes	Important indexes
Constraints	Uniqueness, enums
Data Lifecycle	Soft-delete, append-only
Data Dictionary	Field dictionary
ERD	Mermaid ER diagrams

6. Developer & Deployment

Document	Description
Development Overview	How to work in the monorepo
Prerequisites	Tools required
Local Development	Setup steps
Project Structure	Directory map
Environment Variables	Env catalog (no secrets)
Configuration	Config patterns
Running the Application	Commands
Testing	Test coverage status
Code Quality	Lint / build
Git Workflow	Repo conventions
Docker	Compose services
Deployment	Production deploy
Production Runbook	Day-2 ops
Monitoring	Health checks
Logging	Logging approach
Backup and Restore	Backup status
Disaster Recovery	DR status
Troubleshooting	Common issues
Security	Security notes
Performance	Performance notes

Also see root: [DOCKER.md](#)

7. User

Document	Description
User Guide	End-user overview
Getting Started	First login
Login	Login and SSO

Document	Description
Navigation	Apps and menus
Feature Guides	Feature how-tos
Role Guides	Role-specific guides
Common Tasks	Task checklists
Troubleshooting	User-facing issues
FAQ	FAQ

Audit

- Documentation Audit — coverage, unknowns, gaps, risks

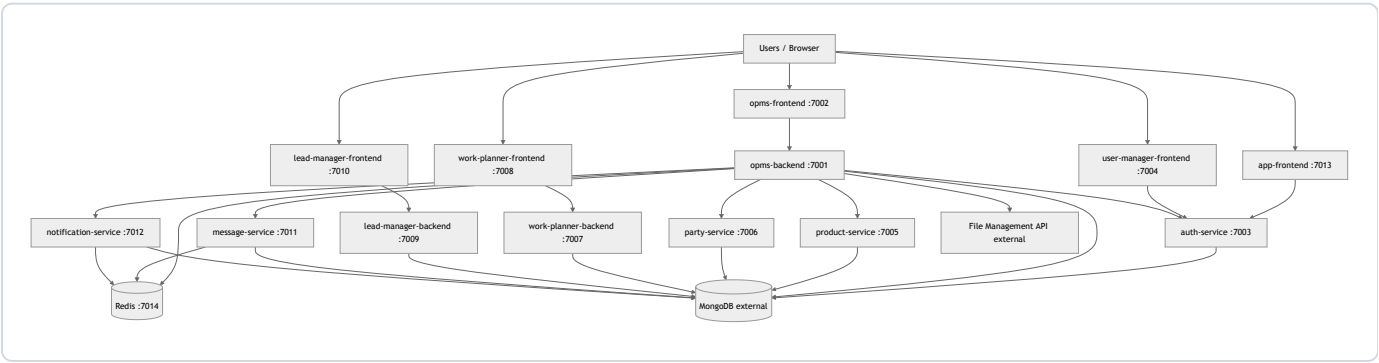
PDF copies

Printable PDFs (Mermaid diagrams rendered) are in [pdf/](#) :

PDF	Contents
OPMS-Documentation-Complete.pdf	Full suite
01-product.pdf	Product
02-architecture.pdf	Architecture + ADRs
03-functional.pdf	Functional
04-api.pdf	API
05-database.pdf	Database
06-developer-deployment.pdf	Developer & deployment
07-user.pdf	User

Regenerate: `cd docs/_pdf-build && node generate-pdf.mjs`

Quick architecture (verified)



Document conventions

Label	Meaning
Verified	Confirmed in source/config
Inferred	Strongly implied by implementation
Unknown / Not identified	Cannot be determined from the repository
Requires confirmation	Needs product/business clarification

Screenshots: **Not available in repository** unless a path is cited.

Documentation Audit

Repository Analyzed

Monorepo at /Users/macbook/Desktop/opms 2 including:

- Frontends: app-frontend , opms-frontend , user-manager-frontend , lead-manager-frontend , work-planner-frontend
- Backends: opms-backend , auth-service , product-service , party-service , lead-manager-backend , work-planner-backend , message-service , notification-service
- Infra: docker-compose*.yaml , DOCKER.md , nginx/ , .env.docker.example
- Existing notes: implementation_plan.md , task.md , opms-backend/docs/**
- Swagger: opms-backend/src/docs/swagger.js , swaggerExtended.js

Documentation Generated

Full suite under docs/ categories 01–07 plus this audit and docs/README.md .

Coverage

Area	Status	Notes
Product	Complete	Based on implemented portals/features; formal product vision Requires confirmation
Architecture	Complete	Diagrams + ADRs evidenced by code
Functional	Complete	Major modules documented; some edge transition matrices partial
API	Complete/Partial	Full inventory in markdown; OpenAPI YAML is inventory subset; richest schemas remain in live /api-docs
Database	Complete/Partial	Catalog + dictionary of key fields; full 100% field dump intentionally summarized with source pointers
Developer/Deployment	Complete	From DOCKER.md/compose/nginx; DR/backup/CI gaps called out
User	Complete	Based on actual routes/nav; no screenshots in repo

Unknowns

1. Official marketed product name / legal branding for all tenants
2. Production whether full compose vs docker-compose.prod.yaml subset is used
3. Exact dual-tenant (spspl + medica) co-hosting procedure
4. Formal RTO/RPO and backup schedules
5. Fine-grained permission codes vs portal roles (Permission model vs live gates)
6. user_manager portal access_roles exact list
7. Full Lead/WorkPlan transition enforcement beyond enums
8. TransportShipment Transporter missing model
9. Automated test strategy and coverage metrics
10. Git branching/PR policy
11. Whether public attachment/terms routes are intentional

Requires Business Confirmation

- Formal PRD vision, MoSCoW priorities, SLAs
- Production portal seeding for `work_planner` , `lead_manager` , `user_manager`
- Support escalation contacts for end users
- Payment/collection lifecycle expectations (essentials docs vs live transitions)
- Whether `closed` order status is still a business requirement

Implementation vs Documentation Gaps

Topic	Code / config says	Other docs say
Order lifecycle	<code>workflow.transitions.js</code> account-inclusive graph	<code>opms-backend/docs/essentials/*</code> older payment/invoice/collection flow — STALE
Order status set	Schema includes <code>closed</code> ; transitions lack <code>closed</code>	Mixed constants (<code>order.constants</code> vs <code>domain.js</code>)
Portal seed	<code>DEFAULT_PORTALS</code> only <code>opms</code>	Apps expect <code>work_planner/lead_manager/user_manager</code>
Swagger server port	<code>swagger.js</code> mentions default 5000	Compose publishes 7001
<code>docker-compose.prod.yml</code>	Partial services	Name implies full production
<code>.env.docker.example</code>	Contains real-looking secrets	Should be placeholders only
<code>OPMS_SCHEMA.md</code>	Mentions fields not in Product schema	<code>mongoRegistry</code> is authority
<code>implementation_plan</code>	Mentions legacy approval route aliases	Current routes center on <code>/api/order-approvals</code> — verify leftover aliases if any

Potential Risks

- Missing CI/CD
- Missing automated backup/DR docs in repo
- Missing centralized monitoring
- Secrets committed in example env file
- Unauthenticated/internal-weak routes (notifications internal, terms, some attachments)
- Redis exposed without auth
- Schema duplication across services (drift)
- OpenAPI docs suite does not fully duplicate every query/body schema (rely on Swagger UI + controllers)
- Stale essentials docs could mislead operators if not marked

Second-pass consistency checks performed

- Port table aligned to `DOCKER.md` / compose defaults (7001–7014)
- Portal role names aligned across product/architecture/user docs
- Workflow transitions sourced from `workflow.transitions.js` not essentials
- Secrets omitted from docs; hygiene warning recorded

Product Overview

Status labels: Content below is **Verified** from the monorepo unless marked otherwise.

Product name

OPMS — Order / Operations Portal Management System (packages also use branding names such as Medica / Shakti in env examples and package names).

Requires confirmation: Official marketed product name and legal entity branding for end customers.

Sources:

- DOCKER.md
- opms-backend/package.json (opms-backend)
- auth-service/package.json (medica-auth-service)
- .env.docker.example (COMPANY_NAME / NEXT_PUBLIC_COMPANY_*)

Purpose

OPMS is a multi-portal business operations platform that enables:

1. **Order lifecycle management** — create, approve, finance/account review, dispatch, transport, delivery, returns, due sheets, flags, reminders
2. **Master data** — parties (customers/suppliers), products (including kits), zones, fleet (vehicles, drivers, transport agents)
3. **CRM** — Lead Manager (leads, follow-ups, quotations, terms)
4. **Field operations** — Work Planner (daily plans, visits, works, expenses)
5. **Identity & access** — Auth service + User Manager (users, departments, roles, portals, company info)
6. **Communications** — in-app notifications, web push, email, WhatsApp messaging

Problem being solved

Inferred from implementation: Organizations need a single identity layer and coordinated portals for sales-to-dispatch order operations, lead-to-quote CRM, and field sales planning — instead of disconnected tools.

Requires confirmation: Exact business problem statement and ROI goals from product/business team.

Target users

Portal / App	Typical users (from code)
App Frontend (SSO hub)	Any authenticated user with department + roles
OPMS	super_admin , admin , sales , finance , account , dispatch
User Manager	Super-admin / admin style access (hasSuperAdminAccess)
Lead Manager	admin , manager , executive
Work Planner	admin , manager , executive (+ global admin bypass)

Sources:

- `opms-backend/src/middlewares/opmsAuth.middleware.js`
- `lead-manager-backend/src/middlewares/leadManagerAuth.middleware.js`
- `work-planner-backend` work planner auth middleware
- Frontends' `authStorage` / `middleware`

Main use cases

1. Log in via App Frontend and launch assigned portals (SSO with `?token=`).
2. Create and progress sales orders through departmental approvals to dispatch/delivery.
3. Maintain party and product masters (and party-product rates).
4. Plan transport and manage fleet/agents.
5. Manage leads, follow-ups, and quotations; convert leads.
6. Plan field work, check in/out visits, submit expenses.
7. Administer users, portal assignments, departments/roles, and company branding.
8. Receive notifications (SSE/in-app/web push) and send transactional messages (email/WhatsApp).

Major capabilities

- JWT authentication with shared `JWT_SECRET`
- Portal-scoped RBAC (`user.portals[].portal_code` + `access_roles`)
- Soft-delete / restore on many domain entities
- BullMQ background jobs (orders, notifications, messages) on Redis
- External file management API for uploads
- Google Sheets webhook ingest (orders, parties, products, reminders)
- Dual-tenant host layout (spspl / medica nginx configs)

Product boundaries

In scope (verified in repo)

- Microservices and micro-frontends listed in `docker-compose.yml`
- Order workflow as implemented in `workflow.transitions.js`
- Lead Manager and Work Planner modules
- Auth, company info, portals, departments
- Messaging and notifications services

Out of scope / not identified in codebase

- Payment gateway / online payment capture — **Not identified** as implemented (payment_status fields exist; collection/paid lifecycle in essentials docs is **stale**)
- Native mobile apps — **Not identified**
- Embedded analytics BI warehouse — **Not identified** (dashboards are API summaries only)
- In-compose MongoDB or MinIO — **external** to Compose
- Automated CI/CD pipelines — **Not identified** (no `.github/workflows`)

Related docs

- [PRD](#)
- [Scope](#)
- [Features](#)
- [System Architecture](#)

Product Requirements Document (PRD)

Source of truth: Implementation in this repository. Numerical SLAs are **not** defined in code unless noted.

Product Vision

Provide a portalized operations suite where a company can authenticate once, assign users to OPMS / Lead Manager / Work Planner / User Manager portals, and execute order-to-delivery, lead-to-quotation, and field-work processes with shared masters and communications.

Requires confirmation: Formal vision statement from product leadership.

Problem Statement

Sales, finance, account, and dispatch teams need coordinated order processing with auditability; sales leadership needs lead pipelines and quotations; field teams need daily work plans and expense capture — under one identity and company branding model.

Objectives

ID	Objective	Evidence
O-1	Support multi-role order workflow with transitions and history	<code>workflow.transitions.js</code> , OrderWorkflow
O-2	Centralize identity and portal access	auth-service User/Portal
O-3	Provide CRM for leads and quotations	lead-manager-*
O-4	Support field work planning and expenses	work-planner-*
O-5	Notify users and send communications	notification-service, message-service
O-6	Deploy as Docker Compose + nginx	<code>docker-compose.yml</code> , <code>nginx/</code>

Target Users

See Personas and Roles.

User Personas

P1 — Sales Executive (OPMS + optional Lead/Work Planner)

- Creates orders, views own pipeline, may manage leads/plans depending on portal assignment.

P2 — Finance / Account Reviewer

- Reviews approvals, due sheets, rates, finance/account queues.

P3 — Dispatch / Logistics

- Creates dispatches, manages transport planner, fleet, deliveries.

P4 — Portal Administrator (User Manager)

- Manages users, portals, departments/roles, company info.

P5 — Lead Manager / Manager

- Oversees team leads, quotations, terms, reports.

P6 — Work Planner Manager

- Assigns team settings, approves plans/expenses.

User Stories

As a sales user,
I want to create and submit an order,
so that finance/account/dispatch can process it.

As a finance user,
I want to approve or reject order approvals,
so that commercial risk is controlled before dispatch.

As a dispatch user,
I want to create dispatches and transport plans,
so that goods can be shipped and tracked.

As any authenticated user,
I want to log into App Frontend and open my assigned portals with SSO,
so that I do not re-authenticate for each app.

As a super admin,
I want to assign portal access_roles to users,
so that each person only sees authorized workspaces.

As a lead executive,
I want to create leads, schedule follow-ups, and convert won leads,
so that the sales pipeline is managed in one place.

As a work planner executive,
I want to create a daily plan with visits and submit expenses,
so that field activity is recorded and approved.

Functional Requirements

ID	Requirement	Source
FR-001	Users can authenticate with email/password and receive JWT	auth-service POST /api/auth/login
FR-002	JWT can be exchanged across frontends via ?token= SSO	app-frontend / portal AuthTokenSsoBootstrap
FR-003	OPMS domain APIs require portal_code opms + access_roles	opmsAuth.middleware.js
FR-004	Orders support create, submit, transition, soft-delete, restore	order.routes.js
FR-005	Order status transitions follow allowed graph	workflow.transitions.js
FR-006	Order approvals support approve/reject/send-to-finance/account/amend	orderApproval.routes.js
FR-007	Dispatches, deliveries, returns, due sheets, unbilled orders are manageable	respective routes
FR-008	Transport planner supports plan lifecycle and order line actions	transportPlanner.routes.js

ID	Requirement	Source
FR-009	Parties and products CRUD (+ soft-delete) via party/product services	party-service, product-service
FR-010	Lead CRUD, follow-ups, convert, quotations, terms	lead-manager-backend
FR-011	Work plans, visits, works, expenses with approve/reject	work-planner-backend
FR-012	Notifications list/stream/read + web push subscribe	notification-service
FR-013	Email/WhatsApp send and communication triggers	message-service
FR-014	Attachments upload via file management API	attachments modules
FR-015	Dashboards per OPMS role	dashboard.routes.js
FR-016	Google Sheet webhooks for orders/parties/products/reminders	webhook routes + secret
FR-017	Company branding via CompanyInfo	auth-service company-info
FR-018	Soft-delete with trash list and restore on many entities	softDelete plugin

Non-Functional Requirements

See [Non-Functional Requirements](#). No latency/availability SLAs found in repository.

Acceptance Criteria (implementation-derived)

Story area	Acceptance criteria (verified behavior)
Login	Valid credentials return JWT; <code>/api/auth/me</code> returns user with portals
OPMS access	User without <code>opms</code> portal receives 403 on domain routes
Order transition	Invalid next status rejected per transitions graph
Soft-delete	Soft-deleted records excluded from default lists; restore returns them
SSO	Opening portal URL with <code>?token=</code> establishes session and strips token
Health	<code>GET /health</code> on backends returns success when process is up

Requires confirmation: Formal UAT scripts and business acceptance sign-off.

Scope

In scope (verified)

Applications

Area	Components
Frontends	app-frontend, opms-frontend, user-manager-frontend, lead-manager-frontend, work-planner-frontend
Backends	opms-backend, auth-service, product-service, party-service, lead-manager-backend, work-planner-backend, message-service, notification-service
Infra in Compose	Redis
Edge	Host nginx configs under <code>nginx/</code> for spspl and medica hostnames
Data store	MongoDB (external URI)
Files	External File Management API (<code>FILE_MANAGEMENT_API_*</code>)

Functional domains

- Authentication, users, departments, roles, portals, company info
- Orders and departmental workflow
- Approvals, finance queues, account steps
- Dispatch, transport shipments, deliveries, returns
- Due sheets, unbilled orders, final order statements
- Flags, reminders, attachments, activity logs
- Fleet: vehicles, drivers, transport agents
- Transport planner
- Party/zone/product catalogs and party-product rates
- Leads, follow-ups, quotations, lead masters, terms
- Work plans, visits, works, expenses, user work-planner settings
- Notifications, web push, email, WhatsApp, auto-emails, communication helpers
- Google Sheets webhook ingestion (selected modules)

Out of scope / not implemented in repo

Item	Status
Compose-hosted MongoDB	Not in compose — external
Compose-hosted MinIO	Leftover <code>minio-data/</code> only; not a service
RabbitMQ	Not present — BullMQ/Redis used
CI/CD pipelines	Not identified
Mobile native apps	Not identified
Full payment collection / invoice_generated lifecycle in live transitions	Essentials docs stale; not in <code>workflow.transitions.js</code>
Automated Mongo backup jobs in repo	Not identified
Centralized APM/metrics stack	Not identified

Partial / dual-tenant

- `docker-compose.prod.yml` defines a **subset** of services — incomplete standalone full deploy (**Verified**).
- Medica tenant uses alternate ports/nginx (5001–5013 / `*.medicaent.in`) via `.env.docker.medica` (**Verified** configs exist; dual-stack runbook **Requires confirmation**).

Related

- [Features](#)
- [Integrations](#)

Personas and Roles

Identity model (verified)

Users are stored in **auth-service** with:

- department (string; seeded values include `super_admin`, `admin`, `sales`, `finance`, `account`, `dispatch`)
- roles[] → Role documents (`sales_executive`, `finance_manager`, etc.)
- portals[{ portal, portal_code, access_roles[] }]

Sources: `auth-service/src/models/User.js`, `Role/Department/Portal` models, seed services.

Portals

portal_code	Seeded in DEFAULT_PORTALS?	access_roles (observed)
opms	Yes	<code>super_admin</code> , <code>admin</code> , <code>sales</code> , <code>finance</code> , <code>account</code> , <code>dispatch</code>
work_planner	No (used in code)	<code>admin</code> , <code>manager</code> , <code>executive</code> (+ <code>super_admin</code> /sales observed in constants)
lead_manager	No (used in code)	<code>admin</code> , <code>manager</code> , <code>executive</code>
user_manager	No (frontend gate)	Unknown exact role list — frontend checks <code>admin/super_admin</code> style access

Requires confirmation: Official portal catalog and which portals must be seeded for production onboarding.

OPMS access roles → UI workspaces

access_role	Frontend portal path	Typical focus (inferred from nav)
<code>super_admin</code>	<code>/super_admin</code>	Full masters + User Manager embed
<code>admin</code>	<code>/admin</code>	Broad order/masters/transport
<code>sales</code>	<code>/sales</code>	Orders overview/create (own data emphasis)
<code>finance</code>	<code>/finance</code>	Finance review, masters, transport
<code>account</code>	<code>/account</code>	Account review, masters, transport
<code>dispatch</code>	<code>/dispatch</code>	Dispatch, transport planner/agents

Source: `opms-frontend` portal nav and middleware.

Lead Manager roles

Role	Capabilities (from UI/API gates)
<code>admin</code>	Full leads; quotations; terms; team reports
<code>manager</code>	Similar to admin for quotations/terms; team oversight
<code>executive</code>	Leads/follow-ups/personal reports; no quotations nav

Work Planner roles

Role	Capabilities
admin	Team manager settings; approve plans/expenses
manager	My team settings; approvals
executive	Own plans, visits, expenses

App Frontend access

Requires JWT with **department** and at least one **role**; sidebar built from `role_codes` ; portal tiles from `portals[]` .

Persona mapping

See PRD personas P1–P6 in [02-prd.md](#).

Features

Feature catalog derived from routes and frontend screens. **Verified** unless noted.

A. App Frontend (SSO Hub)

Feature	Description	Source
Login	Email/password login with company branding	app-frontend /
Department dashboard	Assigned portals launcher	/dashboard/[department]
Profile / password	Change password	/dashboard/profile
Notifications	Bell via auth proxy	NotificationBell
SSO launch	Opens peer apps with ?token=	PortalsSection

B. OPMS Frontend / Backend

Feature	Description
Role workspaces	admin/sales/finance/dispatch/account/super_admin
Order master	List by workflow/priority; create; detail with lifecycle tabs
Order workflow	Submit, transition, history, assignees, fulfillment
Approvals	Unified OrderApproval actions
Finance queues	/api/finance/queue , /summary
Dispatch	Create/patch soft-delete with bill document upload
Deliveries / returns	Log shipment delivery; returns CRUD
Due sheets / unbilled / final statements	Billing support artifacts
Flags	Raise/manage order flags (blocking types in workflow rules)
Reminders	Reminders + follow-ups; Google Sheet webhook
Parties / zones	Via party-service proxy
Products / groups / brands / manufacturers / kits	Via product-service proxy
Fleet	Vehicles, drivers, transport agents
Transport planner	Plans, calendar, LR, packed/dispatched/delivered
Attachments / files	Upload + view/download redirects
Dashboards	Per-role KPI endpoints
Activity log	List activity
Terms & conditions	OPMS T&C API (note: router lacks requireAuth — see security)
Communication	Proxied to message-service
Push notifications	Proxied to notification-service
PDF/export helpers	Frontend html2canvas/jspdf

C. User Manager

Feature	Description
User directory CRUD	List/add/edit/detail
Portals registry	Seed defaults; assign per user
Departments & roles	Hub management
Company information	Branding and legal/tax fields
KPI cards	User counts

D. Lead Manager

Feature	Description
Lead pipeline	CRUD, assign, status, qualify, mark lost, convert
Follow-ups	Calendar + reminders (cron digests)
Reports	Admin "Sales Reports" / others "My Reports"
Quotations	CRUD, submit/approve/reject, PDF, email
Terms & conditions	Master for quotations
Bulk / Google Sheet ingest	Lead import paths
Attachments	Lead attachments module

E. Work Planner

Feature	Description
Work plans	Create/list/calendar; submit/approve/reject/complete
Visits / works	Nested lifecycle (check-in/out/complete)
Expenses	Submit/approve/reject with receipts
Assigned users / settings	Manager templates, CC emails, plan types
Day-end / exports	PDF/Excel helpers in UI

F. Platform services

Feature	Description
Auth	Login, me, change-password
Notifications	Internal create, SSE stream, mark read, web push
Messages	WhatsApp webhook, send message/email, auto-email trigger, communication helpers

Business Requirements

Mapped from implemented behavior. Formal BR IDs from business analysts: **Requires confirmation.**

BR ID	Statement	Implementation evidence
BR-001	System shall authenticate users and issue time-bounded tokens	JWT + JWT_EXPIRES_IN
BR-002	Access shall be portal-scoped	<code>portals[].portal_code</code> + middleware
BR-003	Orders shall follow controlled status transitions	<code>workflow.transitions.js</code>
BR-004	Certain flags shall block dispatch/transit/delivery transitions	<code>workflow.rules.js</code> flag blocks
BR-005	Pricing shall lock after finance/account/dispatch stages	pricing lock statuses in rules
BR-006	Soft-deleted records shall be recoverable by authorized users	softDelete plugin + restore routes
BR-007	Company branding shall be configurable	CompanyInfo module
BR-008	Field teams shall plan daily work and claim expenses	Work Planner models/APIs
BR-009	Sales shall manage leads through quotation to conversion	Lead Manager
BR-010	System shall notify users of relevant events	notification-service + queues
BR-011	System may send email/WhatsApp for order/comms	message-service integrations
BR-012	External sheets may ingest masters/orders via webhook + secret	Google Sheet webhook routes

Priority

Not identified in repository (no MoSCoW tags in code).

Related

- [Business Rules](#)
- [PRD](#)

Non-Functional Requirements

Only document NFRs evidenced in configuration/code. Do not invent numeric SLAs.

Security

NFR	Status	Evidence
JWT authentication	Implemented	auth-service, shared JWT_SECRET
Portal RBAC	Implemented	middleware per portal
Password hashing	Implemented	bcryptjs
HTTPS at edge	Configured for prod hosts	nginx ssl snippets
Secrets via env	Documented	<code>.env.docker</code> , DOCKER.md
Break-glass master password	Optional env	MASTER_PASSWORD — leave empty in prod (DOCKER.md)
Rate limiting	Not identified as middleware	—
Redis AUTH	Not configured in compose	security risk

Performance

NFR	Status
JSON body limit configurable	JSON_BODY_LIMIT (example 50mb)
Nginx proxy timeouts	API hosts use extended timeouts (e.g. 120s) and large body (50m) on api-opms
Explicit p95 latency targets	Not identified

Availability / Scalability

NFR	Status
Container restart policy	restart: unless-stopped
Horizontal scaling playbooks	Not identified
Redis health dependency for queues	message/notification/opms-backend

Reliability

NFR	Status
Health endpoints	/health on backends
BullMQ retry/attempts	Message model tracks attempts; exact policies Requires confirmation from queue configs
Soft-delete over hard-delete	Many domain models

Maintainability

NFR	Status
Modular Express modules	<code>src/modules/*</code>
OpenAPI for opms-backend	<code>/api-docs</code>
Shared schema registries duplicated across services	Maintainability risk — documented in audit

Auditability

NFR	Status
OrderWorkflow / OrderStatusHistory	Append-only workflow audit
ActivityLog	Present
Who approved (signatures on OrderApproval)	Implemented

Accessibility / i18n

Not identified as systematic requirements in frontend code.

Related

- Security
- Performance

Business Rules

Order status transitions (verified)

Source: opms-backend/src/modules/workflow/workflow.transitions.js

```
draft → submitted, cancelled
submitted → sales_approved, finance_review, finance_rejected, on_hold, cancelled
sales_approved → finance_review, finance_rejected, on_hold, cancelled
finance_review → finance_approved, finance_rejected, on_hold, dispatch, cancelled
finance_rejected → submitted, cancelled
finance_approved → account_review, dispatch, on_hold, finance_rejected, cancelled
account_review → account_approved, account_rejected, finance_rejected, on_hold, cancelled
account_rejected → account_review, finance_approved, cancelled
account_approved → dispatch, account_review, on_hold, finance_rejected, account_rejected, cancelled
dispatch → in_transit, on_hold, finance_rejected, cancelled
in_transit → delivered, on_hold, cancelled
delivered → in_transit, cancelled
on_hold → (resume to multiple prior active statuses), cancelled
cancelled → (none)
```

Note: Schema includes `closed` but transition graph has **no** `closed` edges. `POST .../close-after-full-delivery` exists — behavior should be verified against service code when documenting close semantics.

Department permission to set status

Source: `workflow.rules.js` (`DEPT_CAN_SET_STATUS` and special cases).

Sales is limited; admin/super_admin broader; finance/account/dispatch scoped.

Flag blocking

Flag types `payment_issue`, `stock_issue`, `dispatch_issue` **block** transitions into `dispatch` / `in_transit` / `delivered`.

Pricing lock

Pricing locked after statuses including `finance_approved`, `account_approved`, `dispatch`, `in_transit`, `delivered` (per rules).

Priority derivation

Order `priority` derived from expected delivery date windows (>10 low, 5–10 normal, 3–4 high, ≤2 urgent) — as documented in schema analysis of Order model.

Soft-delete

Default queries exclude `deletedAt != null`. Restore endpoints reverse soft-delete.

Portal access

Missing portal assignment ⇒ 403 on portal-gated APIs even if `department` is set (swagger description + middleware).

Lead / Work Planner state machines

Enums exist on models; full transition matrices beyond enums: **partially documented** in functional docs — deep service validation **Requires confirmation** for edge cases.

Stale documentation warning

`opms-backend/docs/essentials/*` describes older payment/invoice/collection flows — **do not** treat as live business rules.

Integrations

Integration	Purpose	Config / evidence	Status
MongoDB Atlas / external Mongo	Primary datastore	MONGO_URI / MONGODB_URI	Required
Redis	BullMQ queues + cache backbone	REDIS_URL , compose redis	Required for queue services
File Management API	Uploads / MinIO-backed files	FILE_MANAGEMENT_API_URL , FILE_MANAGEMENT_API_KEY	Optional
SMTP	Email fallback	SMTP_*	Optional
Microsoft Graph	Preferred outbound email	MICROSOFT_GRAPH_*	Optional
WhatsApp Cloud API	Templates + webhooks	WHATSAPP_*	Optional
Web Push (VAPID)	Browser push	VAPID_*	Optional
Google Sheets webhooks	Ingest orders/parties/products/reminders	GOOGLE_SHEET_WEBHOOK_SECRET	Optional
Host nginx + Let's Encrypt	TLS termination	nginx/	Prod edge
Inter-service HTTP	BFF proxies	*_SERVICE_URL	Required in compose

Not identified

- Payment gateways (Razorpay/Stripe/etc.)
- SMS providers beyond WhatsApp
- External ERP connectors (SAP/etc.)
- RabbitMQ / Kafka

Related

- [Integration Architecture](#)

Product Glossary

Term	Meaning in this codebase
OPMS	Core order operations portal and its backend BFF
Portal	Application workspace assigned on a user (<code>portal_code</code>)
access_roles	Role strings within a portal (e.g. <code>finance</code> under <code>opms</code>)
Party	Customer/supplier (or both) master record
Order	Central commercial/fulfillment document with workflow status
OrderApproval	Unified approval document for admin/finance/account signatures
Due sheet	Documented due/commercial sheet attached to an order
Unbilled order	Tracking record for unbilled quantities/status
Dispatch	Warehouse dispatch against an order
Transport plan	Planned multi-order shipment plan for an agent/date
Lead	CRM prospect in Lead Manager
Quotation	Lead quotation with approval/status enums
Work plan	Daily field plan for a sales user
Soft-delete	Record marked with <code>deletedAt</code> rather than removed
SSO hub	app-frontend launching other apps with JWT query token
BFF	Backend-for-frontend pattern — <code>opms-backend</code> proxies sibling APIs
BullMQ	Redis-backed job queue library
CompanyInfo	Company branding and legal/tax profile

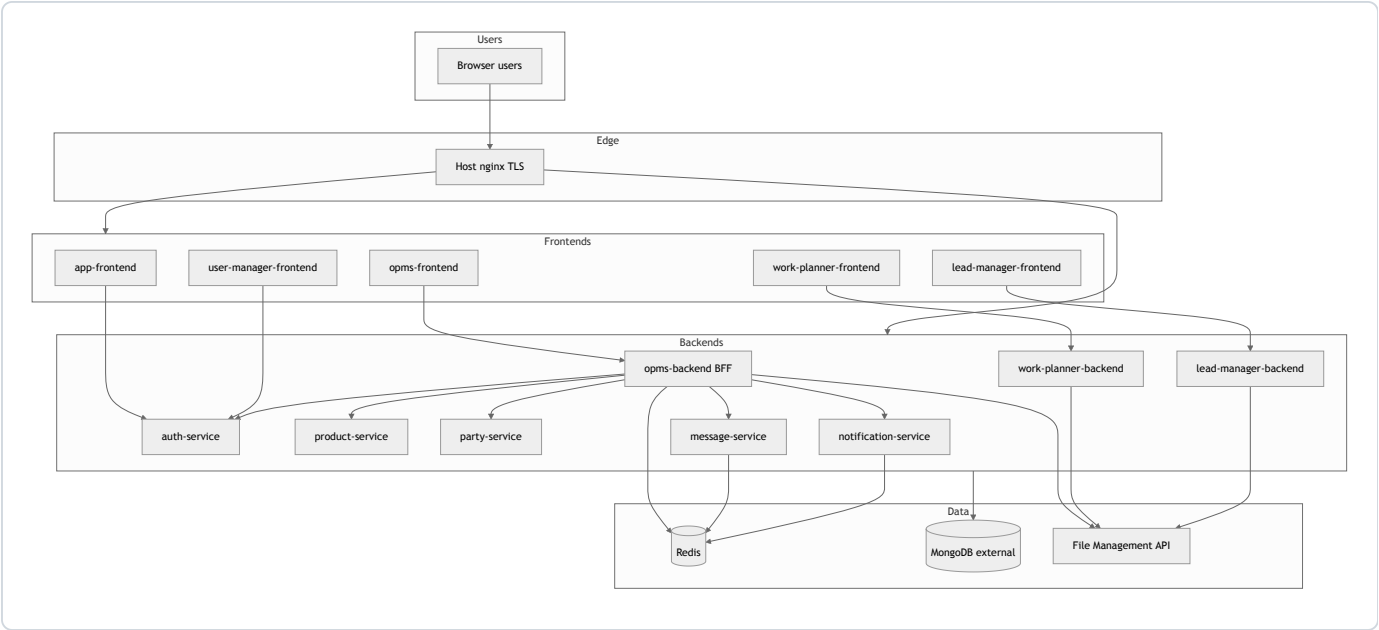
Related

- [Features](#)
- [Database Overview](#)

System Architecture

Context

OPMS is a **Docker Compose—orchestrated microservice + micro-frontend** system. Browsers talk to Next.js apps and/or public API hostnames (via host nginx in production). Backends share a **MongoDB** database (external) and **Redis** (in Compose) for queues.



Runtime ports (Compose defaults)

Service	Port
opms-backend	7001
opms-frontend	7002
auth-service	7003
user-manager-frontend	7004
product-service	7005
party-service	7006
work-planner-backend	7007
work-planner-frontend	7008
lead-manager-backend	7009
lead-manager-frontend	7010
message-service	7011
notification-service	7012
app-frontend	7013
redis	7014→6379

Sources: `docker-compose.yml` , `DOCKER.md`

Architectural style

Style element	Application
Microservices	Domain-split Node/Express services
BFF / API gateway-lite	opms-backend proxies auth/product/party/message/notification
Micro-frontends	Separate Next.js apps; SSO hub launches peers
Shared database	Multiple services point at same Mongo URI (Inferred from env sharing; collection ownership logical)
Async messaging	BullMQ on Redis (not RabbitMQ)

Related

- [Component Architecture](#)
- [Deployment Architecture](#)

Component Architecture

Backend components

Component	Responsibility
auth-service	Login, JWT, users, roles, departments, portals, company info; seeds defaults
opms-backend	Order domain, workflow, finance/dispatch/fleet/transport planner, dashboards; proxies sibling APIs; BullMQ workers
product-service	Product catalog, groups/subgroups/brands/manufacturers, kits, batches
party-service	Parties, zones, party-product mappings/rates
lead-manager-backend	Leads, quotations, lead masters, follow-up cron; proxies messaging
work-planner-backend	Work plans, visits, works, expenses, settings; proxies parties/leads/messaging
message-service	Email/WhatsApp/messages queues
notification-service	In-app notifications, SSE, web push

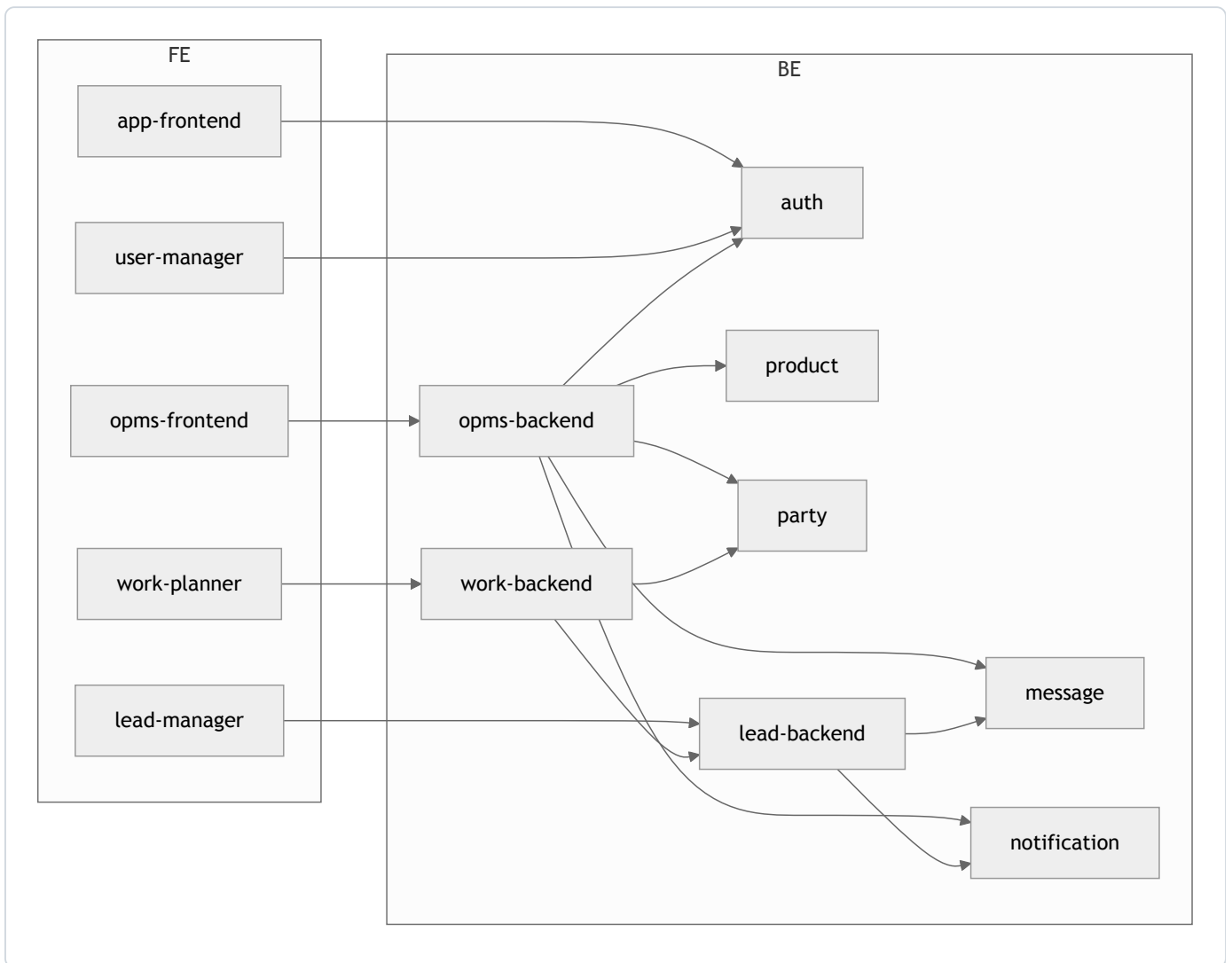
Frontend components

Component	Responsibility
app-frontend	Login + portal launcher SSO hub
opms-frontend	Multi-role OPMS workspace
user-manager-frontend	Identity administration UI
lead-manager-frontend	CRM UI
work-planner-frontend	Field planning UI

Shared technical components

- Mongoose models / `mongoRegistry.js` (duplicated across services)
- `softDelete.plugin.js`
- JWT auth middlewares
- Multer uploads → File Management API
- Swagger (opms-backend only)

Component diagram

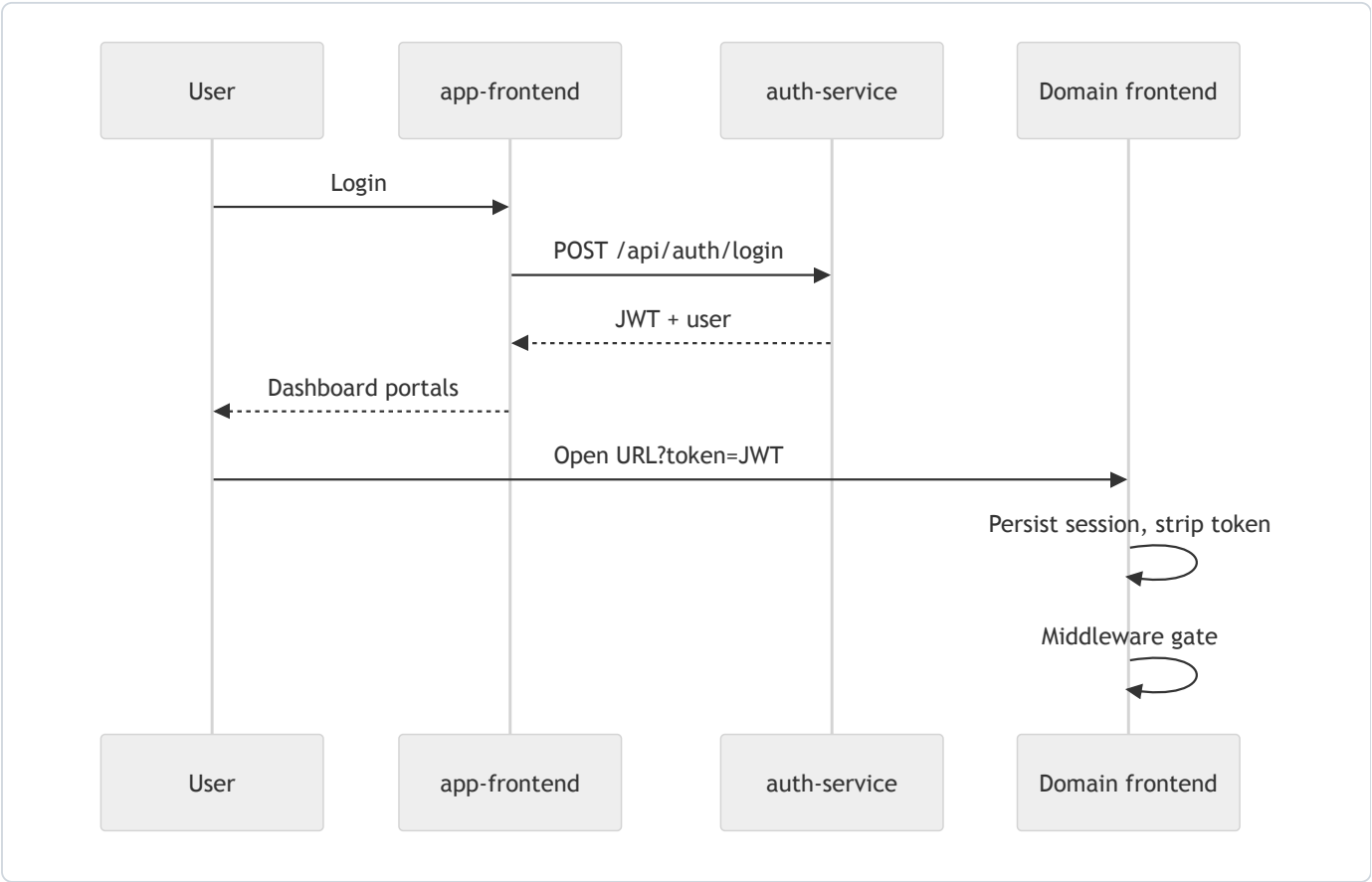


Frontend Architecture

Stack (all five apps)

- **Next.js 16** App Router
- **React 19**
- **Redux Toolkit** + RTK Query
- **Tailwind CSS 4**
- **lucide-react, sonner**

Hub-and-spoke



Session storage patterns

App	localStorage key (primary)	Cookies
app-frontend	shakti.app.session	shakti_session , medica_session
opms-frontend	medica.auth	medica_session , medica_opms_roles
user-manager	shakti.user_manager.session	shared session cookies
lead-manager	shakti.lead_manager.session	shared
work-planner	shakti.work_planner.session	shared

OPMS shell

- Dynamic route `/[portal]/[...rest]`
- Shared Sidebar/Topbar
- Portal allowlists for paths
- Deep links to Lead Manager / User Manager iframe for some paths

API clients

Frontend	Primary base
app-frontend	NEXT_PUBLIC_AUTH_SERVICE_URL
opms-frontend	NEXT_PUBLIC_API_ORIGIN → opms-backend
user-manager	auth-service URL
lead-manager	NEXT_PUBLIC_LEAD_MANAGER_SERVICE_URL
work-planner	NEXT_PUBLIC_WORK_PLANNER_SERVICE_URL

Note: NEXT_PUBLIC_* values are **build-time** baked into images.

Backend Architecture

Common pattern

Each API service:

1. `src/server.js` — connect Mongo, optional queues/workers, listen
2. `src/app.js` — Express app, CORS, JSON limit, mount routers, `/health`
3. `src/modules/*` — routes → controllers → services
4. `src/data/mongoRegistry.js` or `src/models/*` — Mongoose schemas
5. `src/middlewares/*` — auth / portal gates

opms-backend as BFF

Mounts local domain routers under `/api/*` and **HTTP-proxies**:

- `/api/auth`, `/api/users`, `/api/company-info`, `/api/portals`, `/api/departments` → auth-service
- `/api/products`, `product-*` → product-service
- `/api/parties`, `zones`, `party-*` → party-service
- `/api/notifications`, `push` → notification-service
- `/api/messages`, `emails`, `auto-emails`, `communication` → message-service

Source: `opms-backend/src/app.js`

Queues / workers (opms-backend)

BullMQ queues under `src/queues/` (orders, orderApproval, dispatches, workflow, reports) with workers in `src/workers/`.

message-service: `messages`, `autoEmails`

notification-service: `notifications`

Auth middleware pattern

1. Global optional JWT populate
2. Route `requireAuth`
3. Portal gate (`requireOpmsAccess` / lead / work planner)

OpenAPI

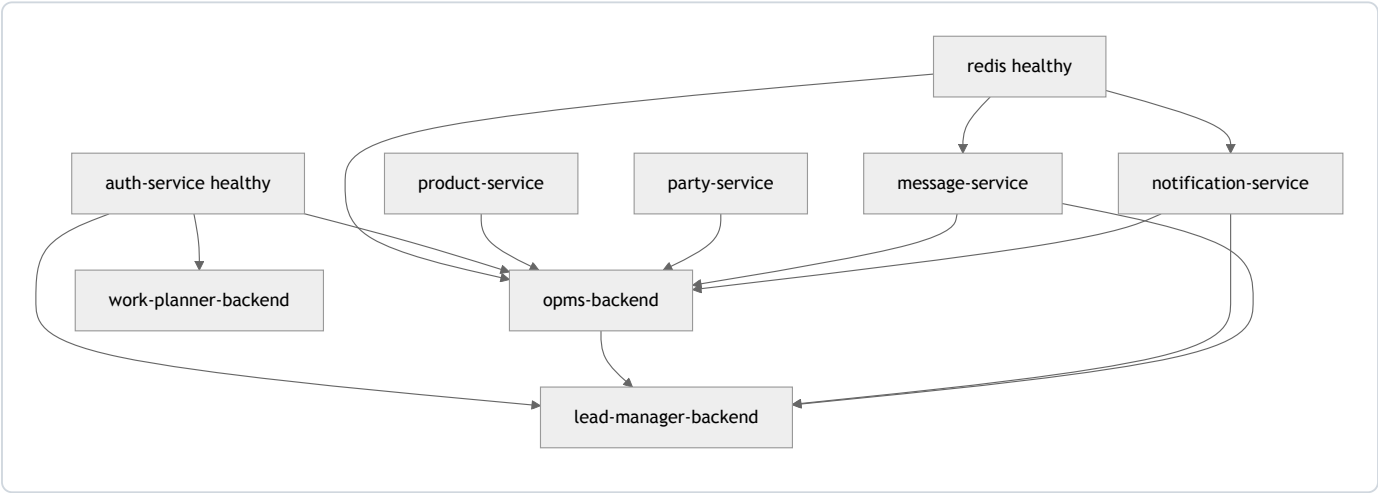
Only opms-backend exposes Swagger UI at `/api-docs`.

Service Architecture

Service communication matrix

From	To	Mechanism
Frontends	Owning backends	HTTPS/HTTP REST + Bearer JWT
opms-backend	auth/product/party/message/notification	HTTP proxy / axios
work-planner-backend	party, lead-manager, message, notification	HTTP proxy
lead-manager-backend	message, notification	HTTP proxy
auth-service	notification-service	proxy /api/notifications
Queue producers	Redis	BullMQ
All APIs	MongoDB	Mongoose
Upload flows	File Management API	HTTP + API key

Startup dependencies (compose)

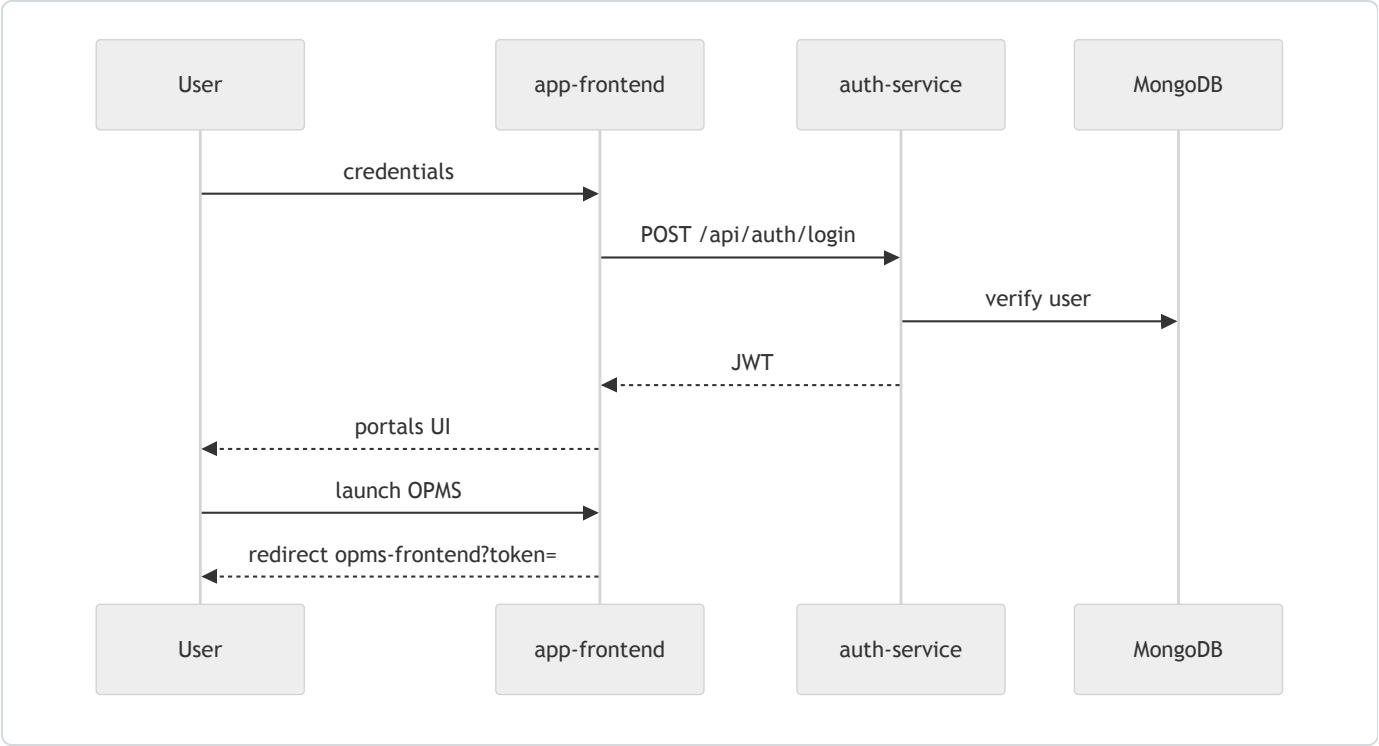


Health

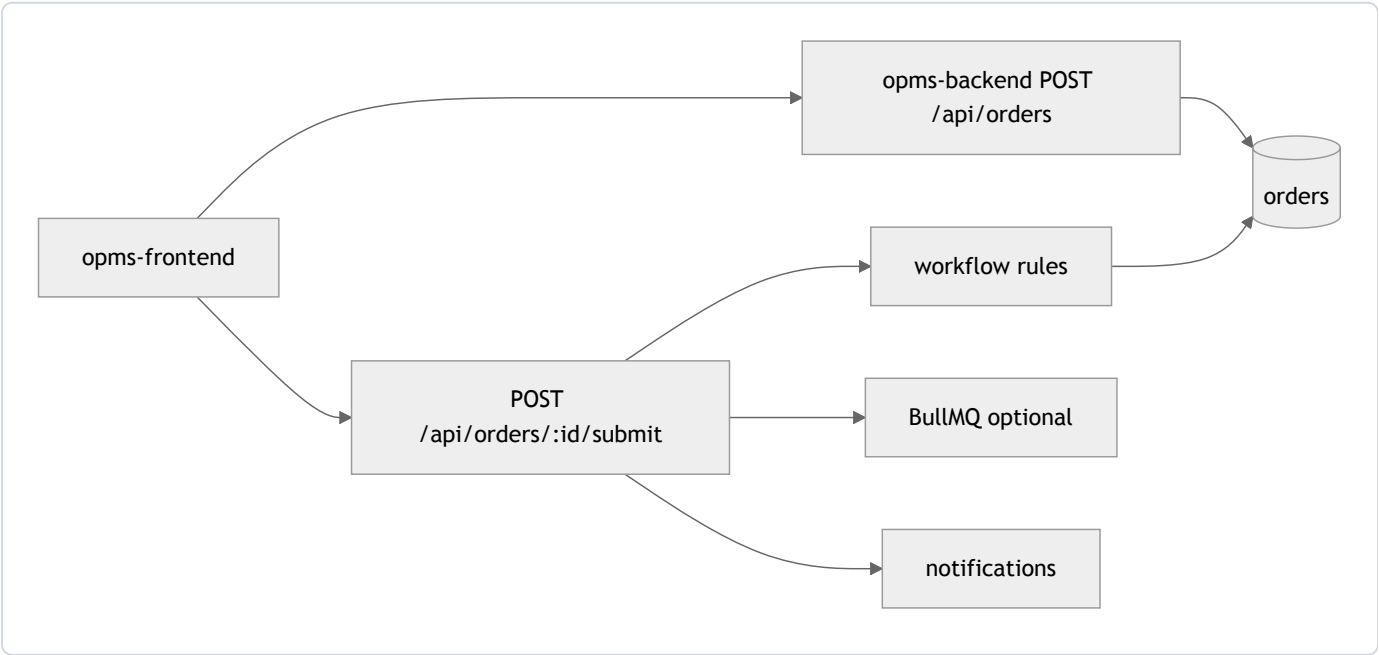
Each backend exposes `GET /health`. Frontends: **no** compose healthchecks in base file.

Data Flow

1. Login and portal launch



2. Create and submit order



3. Approval path (simplified)

Sales/Admin create/update OrderApproval → send-to-finance → finance approve/amend → send-to-account → account approve → order status advances per transitions → dispatch created → transport/delivery.

4. Messaging

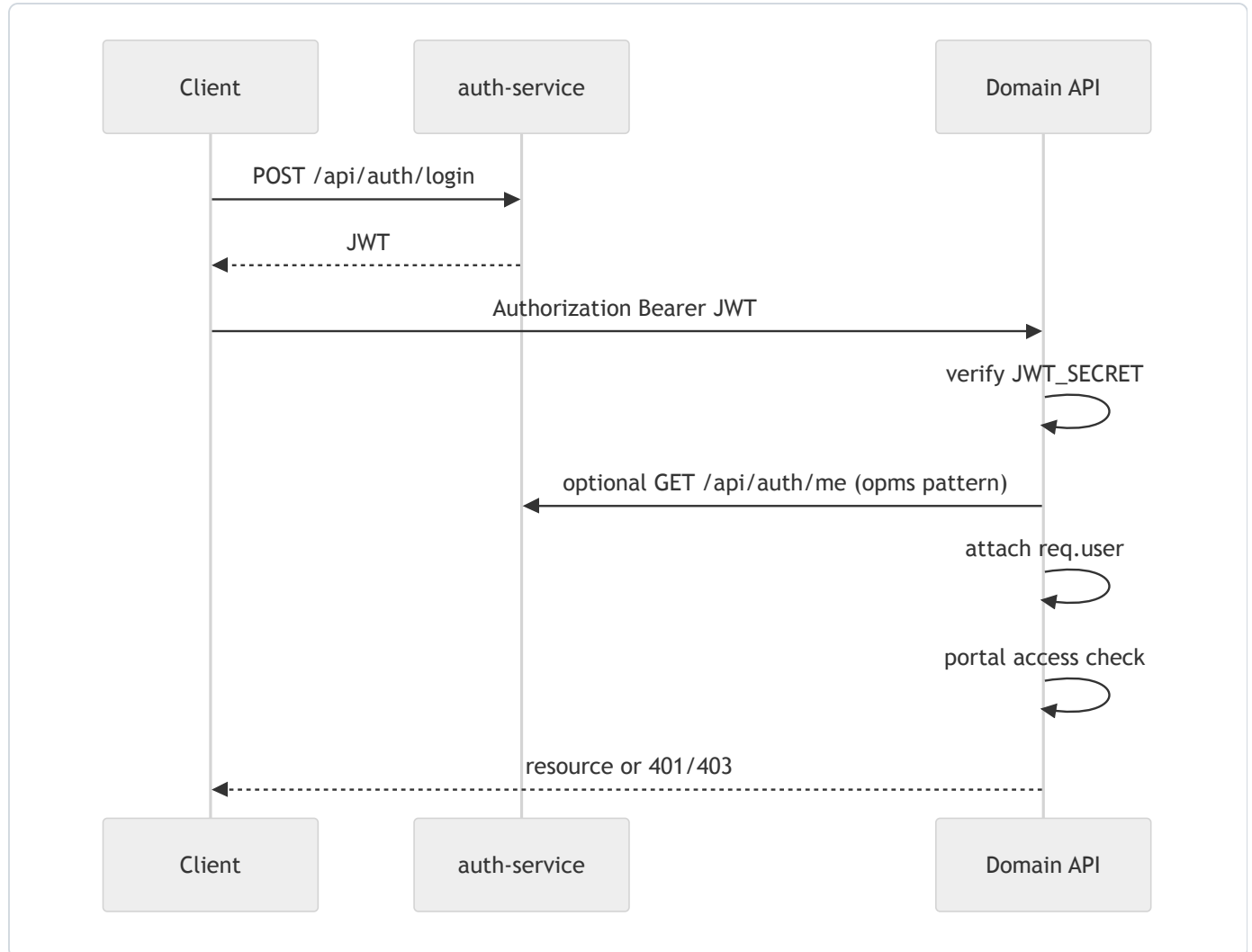
Domain event/API → message-service enqueue → worker sends via Graph/SMTP/WhatsApp → Message document status updated.

5. File upload

Client → attachments route (multer) → File Management API → Attachment metadata in Mongo → view/download via `/api/files/:fileId/*` redirect.

Authentication & Authorization

Authentication flow



Token

- Signed with `JWT_SECRET`
- Expiry `JWT_EXPIRES_IN` (example default `7d`)
- Optional `MASTER_PASSWORD` break-glass (**dangerous if set in prod**)

Authorization layers

1. **Authenticated** — valid JWT
2. **Portal membership** — `portals[]` contains required `portal_code`
3. **access_roles** — role string must match portal gate (or admin bypass where coded)
4. **Handler-level checks** — department / role for specific actions (workflow rules, lead admin routes)
5. **Soft-delete permission** — authenticated (+ opms) for trash/restore on some routes; swagger mentions permission codes that may be aspirational vs portal model — **Requires confirmation** for fine-grained permission codes vs portal roles

OPMS roles

super_admin | admin | sales | finance | account | dispatch

Cross-portal

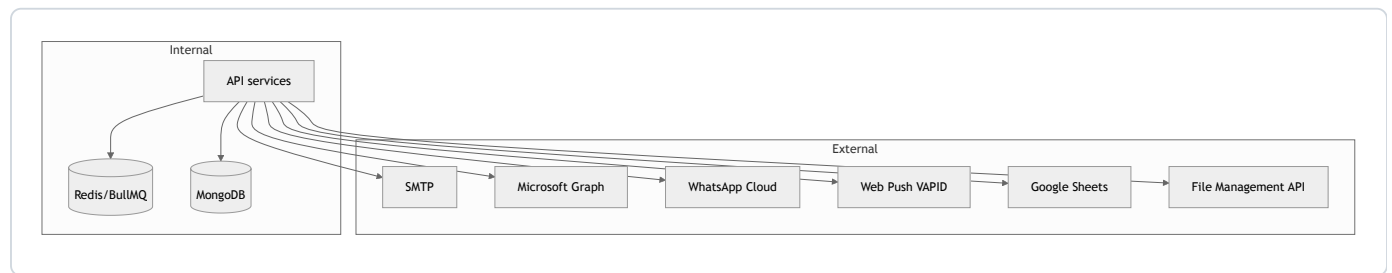
Lead list may allow work_planner access in some routes (`requireLeadManagerOrWorkPlannerAccess`).

Public / weakly protected surfaces (document carefully)

Surface	Notes
POST /api/auth/login	Public
GET /api/company-info	Public on auth-service
Google Sheet webhooks	Secret header/param
WhatsApp webhooks	Public verify/receive
notification internal POSTs	No requireAuth — service trust
opms terms-and-conditions router	No requireAuth on router
lead attachments router	No requireAuth on router
work-planner attachment view	Public view path before auth

See [Security](#).

Integration Architecture



Email

Preferred: Microsoft Graph sendMail when configured; SMTP fallback via nodemailer.

WhatsApp

Cloud API templates + webhook endpoints on message-service.

Files

All binary storage delegated to external File Management API (historically MinIO-backed). Local `minio-data/` is leftover, not compose-managed.

Sheets

Inbound webhooks protected by `GOOGLE_SHEET_WEBHOOK_SECRET`.

Infrastructure Architecture

Compose topology

Sources: `docker-compose.yml` , `docker-compose.dev.yml` , `docker-compose.prod.yml` , `DOCKER.md`

- Single default bridge network
- Named volume: `redis_data`
- `extra_hosts`: `host.docker.internal:host-gateway` for host Mongo
- Images: `node:20-alpine`

External dependencies

- MongoDB (Atlas or host)
- File Management API
- Optional SaaS: Graph, WhatsApp, SMTP

Nginx

Host-installed reverse proxy; configs in `nginx/sites-available/opms` and `medica` .

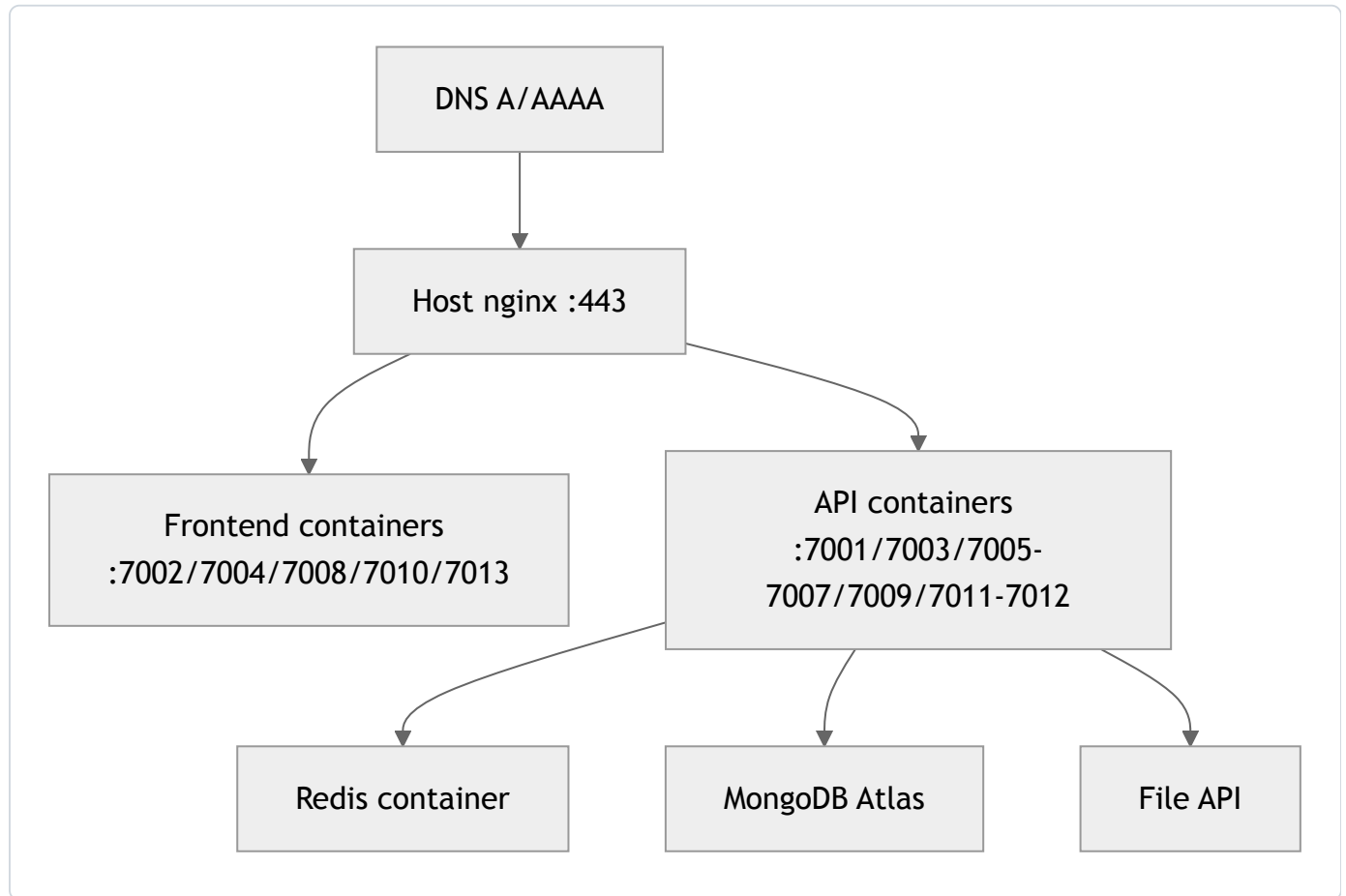
Dual tenant

Tenant	FE/API ports	DNS
OPMS/spspl	7001–7014	<code>*.spspl.com</code>
Medica	5001–5013 (nginx upstreams)	<code>*.medicaent.in</code>

What is not in infra-as-code here

- CI/CD
- Kubernetes/Helm
- Managed monitoring
- Mongo/Redis backup automation

Deployment Architecture



Recommended path (from DOCKER.md)

1. Configure `.env.docker`
2. `docker compose --env-file .env.docker up --build -d`
3. Install nginx site + certbot SANs
4. Point DNS to host

Prod compose file caveat

`docker-compose.prod.yml` is a **partial** stack — prefer full `docker-compose.yml` unless ops intentionally runs a subset (**Verified** by file contents).

Rollback

Not identified as automated. Typical approach: redeploy previous image tags / git revision and `compose up` — **Requires confirmation** of image registry strategy (compose builds locally by default).

Architecture Decisions (Index)

Historical rationale is often **not recorded** in-repo. ADRs below capture decisions **evidenced by the current implementation**, with unknown history called out.

ADR	Title
ADR-001	Split domain into multiple Express services
ADR-002	opms-backend as BFF/proxy
ADR-003	Shared MongoDB across services
ADR-004	JWT + portal access_roles RBAC
ADR-005	BullMQ on Redis instead of broker
ADR-006	Micro-frontends with SSO hub
ADR-007	External file management API
ADR-008	Unified OrderApproval model

ADR-001 — Microservice split by domain

Status: Accepted (evidenced by repo structure)

Date: Not identified in repository

Context: Large order/CRM/field/ops domain with multiple UIs.

Decision: Separate Node services: auth, opms, product, party, lead-manager, work-planner, message, notification.

Alternatives: Modular monolith; serverless functions — **Not evidenced**.

Consequences: Independent deploy/scale; duplicated mongoose registries; operational complexity.

Evidence: Top-level service directories + compose services.

ADR-002 — opms-backend as BFF/proxy

Status: Accepted (evidenced)

Date: Not identified

Context: OPMS UI needs one API origin for many resources.

Decision: Proxy auth/product/party/message/notification under opms-backend `/api/*`.

Alternatives: Frontend calls each service directly only — partially used by other MFEs.

Consequences: Single CORS/API origin for OPMS; coupling/latency through proxy.

Evidence: `opms-backend/src/app.js`

ADR-003 — Shared MongoDB

Status: Accepted (inferred from shared URI + mirrored registries)

Date: Not identified

Context: Cross-service references (User, Party, Order, Lead).

Decision: Services connect to same Mongo deployment; schemas registered per service.

Alternatives: DB-per-service with sync APIs — not implemented.

Consequences: Simple joins by ObjectId; schema drift risk across copied registries.

Evidence: Shared `MONGO_URI` in compose env; parallel `mongoRegistry.js` files.

ADR-004 — JWT with portal access_roles

Status: Accepted

Date: Not identified

Context: Multi-app access control beyond department alone.

Decision: Authorize via `user.portals[].portal_code + access_roles`.

Alternatives: Permission-code RBAC only (Permission model exists in opms registry but live gates are portal-centric).

Consequences: Flexible multi-portal users; seed gap for non-opms portals.

Evidence: Portal middlewares across services.

ADR-005 — BullMQ on Redis

Status: Accepted

Date: Not identified

Context: Async notifications/messages/order jobs.

Decision: Use Redis + BullMQ; do not run RabbitMQ.

Alternatives: RabbitMQ/Kafka — absent.

Consequences: Simple compose dependency; Redis becomes critical path.

Evidence: redis service; bullmq dependencies; queue folders.

ADR-006 — Micro-frontends with SSO hub

Status: Accepted

Date: Not identified

Context: Distinct UX domains (ops, CRM, plans, admin).

Decision: Separate Next apps; app-frontend launches with `?token=` .

Alternatives: Single SPA with route modules — not chosen.

Consequences: Independent releases; duplicated shell/auth utilities; cookie/localStorage key proliferation.

Evidence: Five frontend packages + SSO bootstrap components.

ADR-007 — External file management API

Status: Accepted

Date: Not identified

Context: Attachments for orders/leads/expenses.

Decision: Store binaries via external File Management API; persist metadata in Mongo Attachment.

Alternatives: In-compose MinIO (leftover data dir only).

Consequences: Decoupled storage; runtime dependency on external API key/URL.

Evidence: `FILE_MANAGEMENT_API_*` ; files routes.

ADR-008 — Unified OrderApproval

Status: Accepted

Date: Referenced in `implementation_plan.md`

Context: Separate admin/finance approval models were hard to maintain.

Decision: Single `OrderApproval` (+ `OrderAmendmentUser`) with signature flags.

Alternatives: Keep separate collections — rejected per plan.

Consequences: Cleaner schema; API surface consolidated under `/api/order-approvals`.

Evidence: `implementation_plan.md`; `orderApproval` module; `mongoRegistry`.

Functional Documentation Index

Module	Doc
Authentication	authentication.md
Users & Identity	users-and-identity.md
Orders & Workflow	orders-and-workflow.md
Approvals & Finance	approvals-and-finance.md
Dispatch & Transport	dispatch-and-transport.md
Parties & Products	parties-and-products.md
Leads & Quotations	leads-and-quotations.md
Work Planner	work-planner.md
Notifications & Messages	notifications-and-messages.md
Attachments & Files	attachments-and-files.md
Dashboards & Activity	dashboards-and-activity.md

Module: Authentication

Purpose

Authenticate users, issue JWTs, expose current user profile, and support password change.

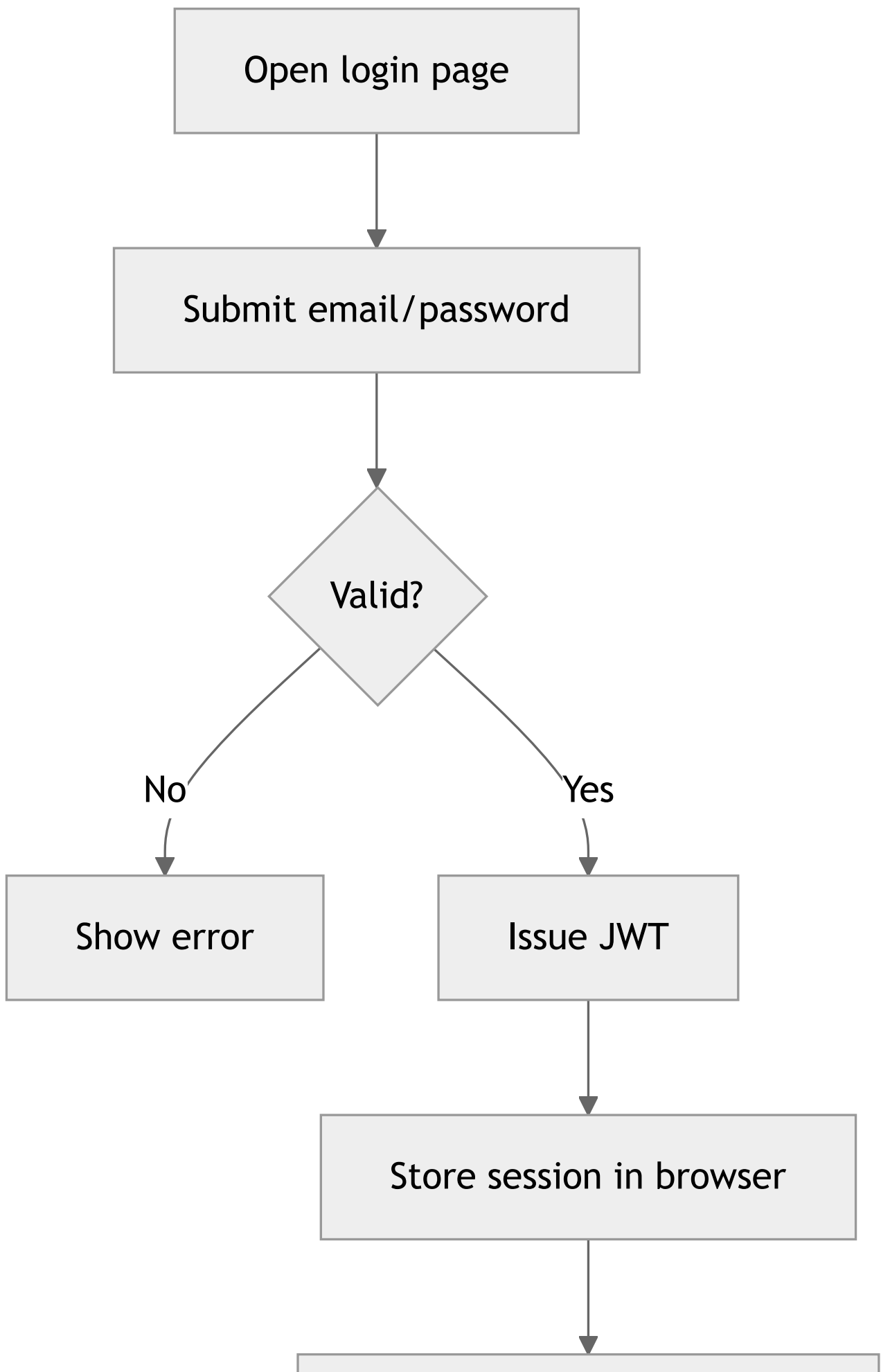
Actors

Any user with an active auth-service account; App Frontend and all portal frontends.

Preconditions

- auth-service running and Mongo reachable
- User exists with password hash; `is_active` true (**Inferred** inactive users rejected — verify service)

Workflow



```
graph TD; A[Navigate dashboard/portal] --> B[Optional SSO ?token= to other app];
```

Navigate dashboard/portal

Optional SSO ?token= to
other app

Business Rules

- Shared `JWT_SECRET` across services
- Optional `MASTER_PASSWORD` can authenticate as any user if set — **must be empty in production**

Validations

Email/password required on login (controller-level).

Permissions

Login public; `/me` and change-password require auth.

Exceptions / Errors

401 invalid credentials; 401/403 on protected routes without/with wrong token.

Related APIs

- `POST /api/auth/login`
- `GET /api/auth/me`
- `POST /api/auth/change-password`

Related DB

User, Role, Portal, Department, CompanyInfo (auth-service)

Related UI

app-frontend `/`, portal `/login` pages

Dependencies

auth-service; MongoDB; frontends

Source

```
auth-service/src/modules/auth/
```

Module: Users & Identity

Purpose

Administer users, departments, roles, portals, and company branding.

Actors

User Manager admins / super admins; auth-service authenticated callers.

Preconditions

Authenticated session with User Manager access gate.

Workflow — assign portal

1. Open User Manager → Portals or User detail
2. Select user
3. Assign `portal_code` + `access_roles`
4. User relaunches App Frontend / portal to pick up JWT claims (**Inferred:** may need re-login)

Business Rules

- Email unique
- DEFAULT_PORTALS seed currently includes **only** `opms`
- Other portals used by apps must be created/assigned manually or via Portals UI seed extensions — **Requires confirmation** of production seed procedure

Permissions

User Manager frontend `hasSuperAdminAccess` ; API routes behind `requireAuth` (finer permission codes **Unknown** vs portal checks)

Related APIs

`/api/users` , `/api/roles` , `/api/portals` , `/api/departments` , `/api/company-info`

Related DB

User, Role, Department, Portal, CompanyInfo

Related UI

user-manager-frontend `/dashboard/*`

Source

`auth-service/src/modules/users|portals|departments|companyInfo/` `user-manager-frontend/`

Module: Orders & Workflow

Purpose

Create and progress sales orders through departmental statuses to delivery/cancellation.

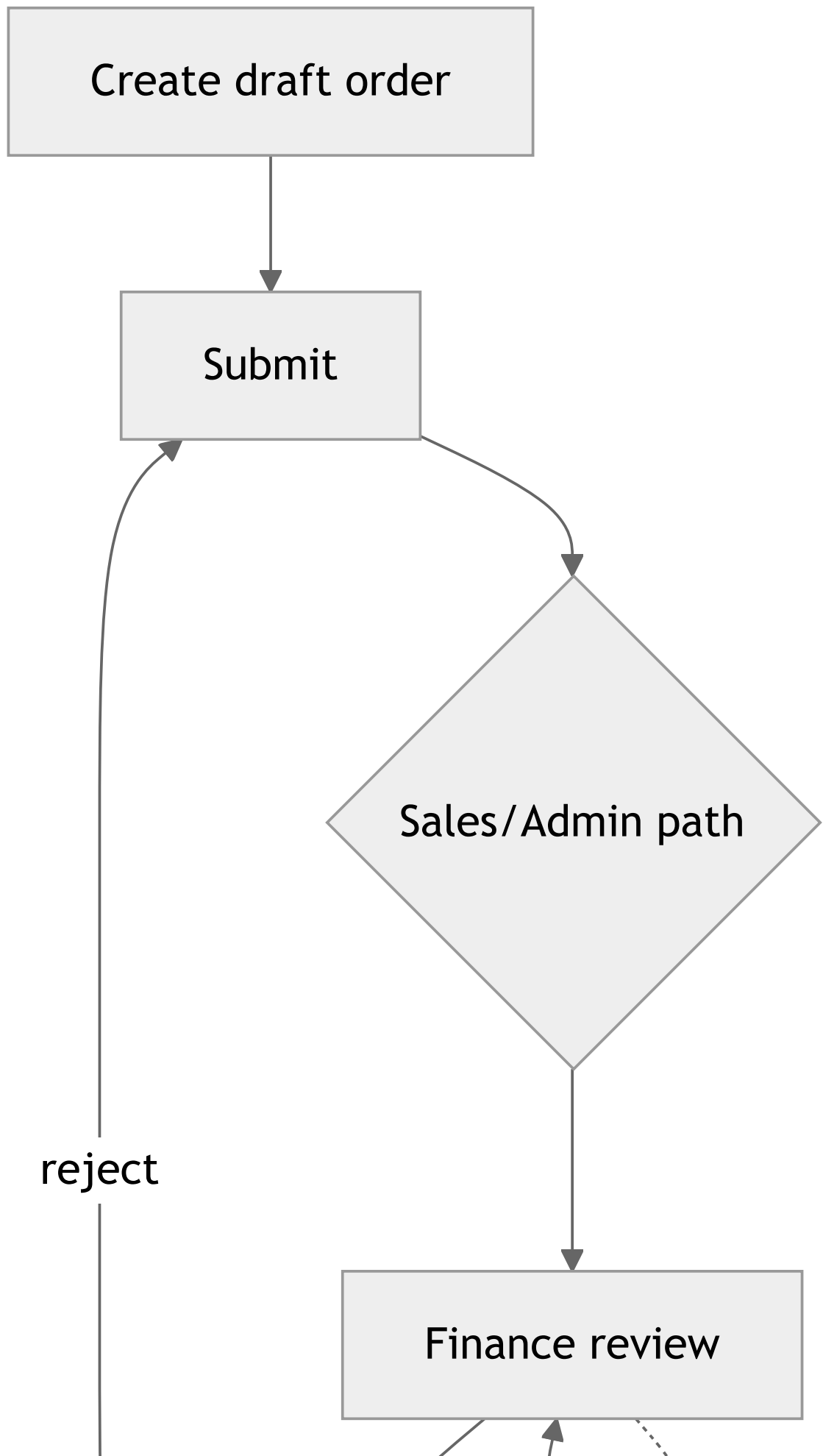
Actors

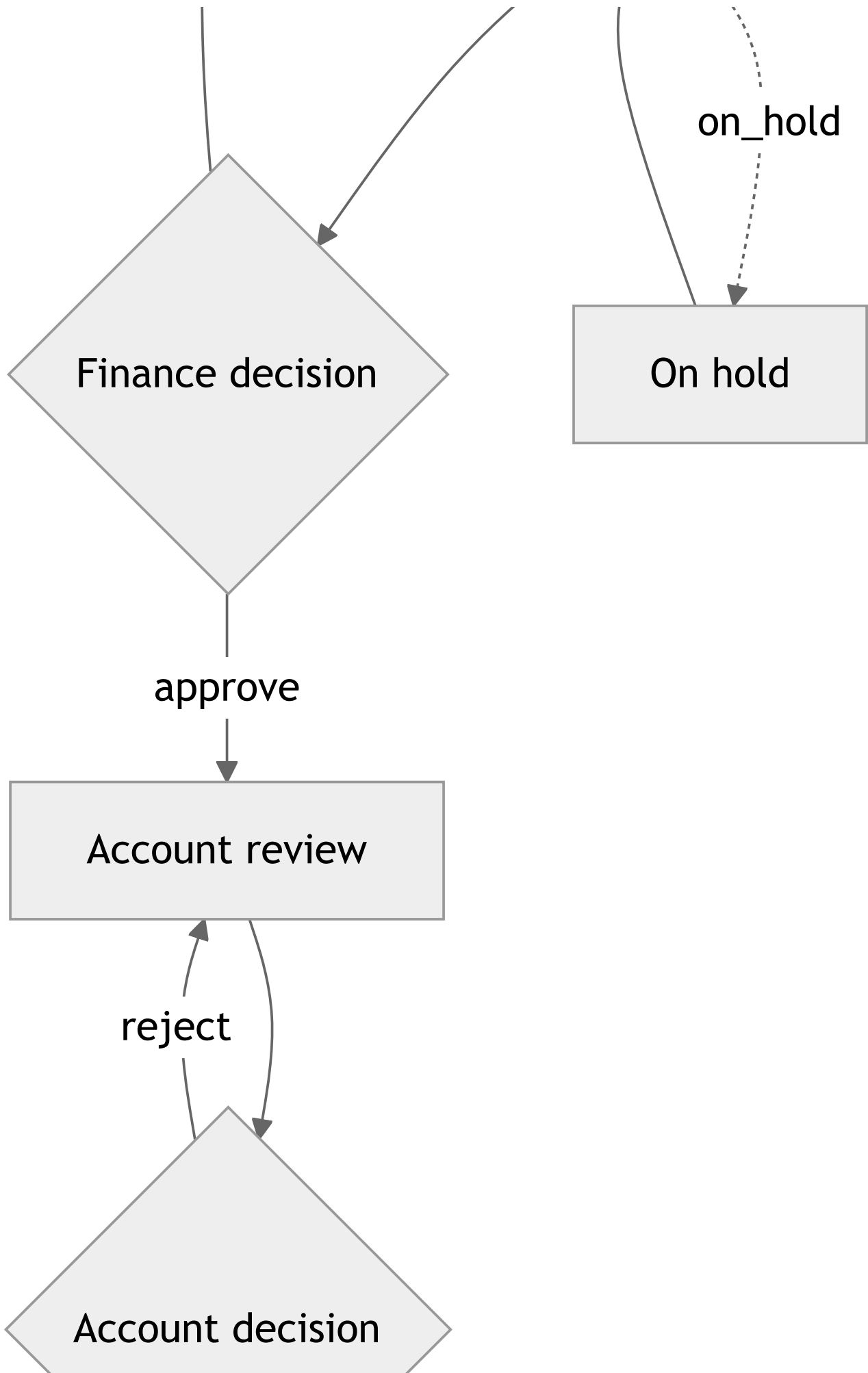
OPMS roles: sales, admin, finance, account, dispatch, super_admin

Preconditions

- OPMS portal access
- Party/product masters available for line items

Workflow





Exact edges: see [Business Rules](#).

Business Rules

- Transition graph enforced
- Flag types can block dispatch/transit/delivery
- Pricing lock after certain statuses
- Soft-delete supported

Validations

Order payload validations in order service/controller (**see service for required fields**).

approve

Permissions

`requireAuth + requireOpmsAccess` transition permission by department rules.

Dispatch

State Transitions

See `Order.status` enums and `workflow.transitions.js`.

In transit

Related APIs

`/api/orders/*`, `/api/orders/:id/transition`, `/submit`, `/close-after-full-delivery`

Delivered

Related DB

`Order`, `OrderWorkflow`, `OrderStatusHistory`, `order_items` embedded

Related UI

`opms-frontend` `/[portal]/orders`, `create-order`, `order/[id]`

Dependencies

`party/product` data; notifications/messages optional

Source

`opms-backend/src/modules/orders/`, `workflow/`

Module: Approvals & Finance / Account

Purpose

Unified order approvals with admin/finance/account signatures; finance queues; due sheets; unbilled tracking; final statements.

Actors

admin, finance, account, super_admin (and sales where allowed)

Workflow (approval document)

1. Create/list OrderApproval for order
2. Admin approve / send-to-finance
3. Finance approve / amend / reject
4. Send-to-account
5. Account approve / amend / reject
6. Resolve-dispatch related actions as exposed by routes

Related APIs

- `/api/order-approvals/*` (approve, reject, send-to-finance, send-to-account, finance-amend, amend, resolve-dispatch, soft-delete)
- `/api/approvals` GET
- `/api/finance/queue` , `/api/finance/summary`
- `/api/order-due-sheets/*`
- `/api/unbilled-orders/*`
- `/api/final-order-statements/*`

Related DB

OrderApproval, OrderAmmendmentUser, OrderDueSheet, UnbilledOrder

Related UI

Order detail approval/finance/account tabs in opms-frontend

Source

```
opms-backend/src/modules/orderApproval|finance|orderDueSheet|unbilledOrder|finalOrderStatement|approvals/implementation_plan.md
```

Module: Dispatch & Transport

Purpose

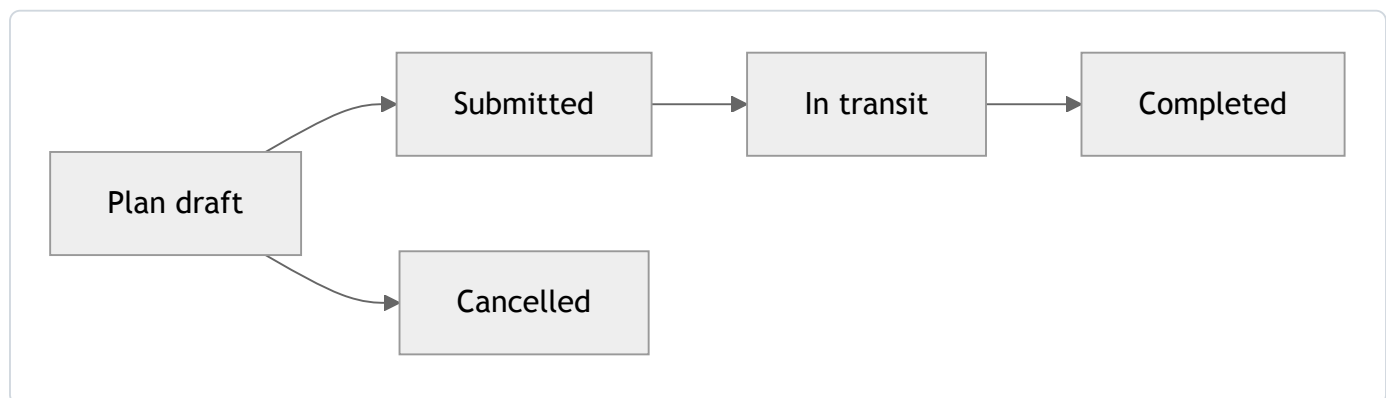
Create dispatches, manage shipments/deliveries/returns, fleet masters, and transport planner.

Actors

dispatch, admin, account, finance, super_admin (per nav/API gates)

Workflow — transport plan

1. List eligible orders
2. Create plan for date/agent
3. Add orders
4. Submit plan
5. Mark packed / generate LR / dispatched / delivered on plan orders
6. Complete or cancel plan



Related APIs

`/api/dispatch`, `/api/transport`, `/api/order-deliveries`, `/api/order-returns`, `/api/vehicles`, `/api/drivers`, `/api/transport-agents`, `/api/transport-plans`

Related DB

OrderDispatch, TransportShipment, OrderDelivery, OrderReturn, Vehicle, Driver, TransportAgent, TransportPlan, TransportPlanOrder

Note

TransportShipment references `Transporter` model not found in registry — **Unknown** / possible schema debt.

Related UI

Dispatch portal; transport planner pages; fleet routes

Source

```
opms-backend/src/modules/dispatch|transport|orderDelivery|orderReturn|fleet|transportPlanner/
```

Module: Parties & Products

Purpose

Maintain commercial masters: parties, zones, products, kits, brands, manufacturers, party-product rates.

Actors

OPMS admin/finance/account/super_admin primarily; APIs require auth (+ department middleware currently effectively auth-only in party/product).

Workflows

- CRUD + soft-delete/restore/bulk operations
- Zone attach parties and sales persons
- Party product mapping + rate approve
- Google Sheet webhook ingest

Related APIs

party-service: `/api/parties` , `/api/zones` , `/api/party-products` , `/api/party-order-products-rate`
product-service: `/api/products` , `/api/product-groups` , `/api/product-subgroups` , `/api/product-brands` ,
`/api/product-manufacturers` , `/api/product-kit-items`
Also reachable via opms-backend proxy.

Related DB

Party, Zone, Product, ProductKitItem, ProductGroup, ProductSubgroup, ProductBrand, ProductManufacturer, Batch, PartyProductMapping, PartyProductRate, PartyProductLastRate

Related UI

opms-frontend Party Master / Product Master sections

Source

`party-service/` , `product-service/`

Module: Leads & Quotations

Purpose

CRM pipeline from lead creation through follow-ups, quotations, and conversion.

Actors

lead_manager admin|manager|executive ; limited cross-access from work_planner for lead list.

Lead statuses (enum)

new , assigned , contacted , qualified , unqualified , follow_up , quotation , negotiation , won , lost , converted

Workflow

1. Create lead
2. Assign / change status / qualify
3. Schedule follow-ups
4. Create quotation (admin/manager)
5. Submit/approve/reject quotation
6. Mark won/lost or convert to party/order linkage fields

Jobs

node-cron follow-up reminder digests (FOLLOWUP_REMINDER_*)

Related APIs

/api/leads , /api/lead-masters , /api/quotations , /api/terms-and-conditions , /api/attachments

Related DB

Lead, LeadFollowUp, LeadSource, LeadLostReason, LeadQuotation, FollowUpDigestLog, Terms*

Related UI

lead-manager-frontend /dashboard/leads|quotations|follow-ups|reports

Source

lead-manager-backend/ , lead-manager-frontend/

Module: Work Planner

Purpose

Daily field plans (visits/works), approvals, expenses, and per-user settings.

Actors

work_planner admin/manager/executive (+ global admin bypass)

WorkPlan statuses

planned , draft , submitted , approved , rejected , completed

Editable (constants): planned , draft , rejected

Workflow

1. Create plan for date (unique per sales_user+date when active)
2. Add visits/works
3. Submit → manager/admin approve/reject
4. Execute visits (check-in/out/complete)
5. Submit expenses → approve/reject
6. Complete plan / day-end flows in UI

Related APIs

/api/work-planner/* (plans, visits, works, expenses, user-settings, uploads, stats)

Related DB

WorkPlan, WorkPlanVisit, WorkPlanWork, WorkPlanExpense, UserWorkPlannerSettings

Related UI

work-planner-frontend dashboard routes

Source

work-planner-backend/ , work-planner-frontend/

Module: Notifications & Messages

Purpose

In-app notifications (incl. SSE), web push, email, WhatsApp, and communication helpers.

Actors

Authenticated users; internal services calling notification internal endpoints.

Notification workflow

1. Domain service calls internal create / order-transition
2. Persist Notification; enqueue push optionally
3. Client lists or SSE streams; mark read

Message workflow

1. API send / communication trigger
2. Enqueue BullMQ job
3. Worker delivers via Graph/SMTP/WhatsApp
4. Update Message status

Permissions

User routes require auth; **internal** notification POSTs are unauthenticated in code — rely on network trust.

Related APIs

notification-service /api/notifications , /api/push/* , /api/subscribe

message-service /api/messages , /api/emails , /api/auto-emails , /api/communication

Related DB

Notification, PushSubscription, Message

Related UI

NotificationBell across frontends; communication tabs in OPMS order UI

Source

notification-service/ , message-service/

Module: Attachments & Files

Purpose

Upload files to external file API; store metadata; view/download via redirects.

Actors

Authenticated OPMS / lead / work-planner users (note weaker auth on some routers — see security)

Workflow

1. Multipart upload to attachments endpoint
2. File Management API stores binary
3. Attachment document saved with `entity_type/entity_id`
4. View/download via `/api/files/:fileId/view|download`

Soft-delete

Supported with restore on OPMS attachments (permission middleware).

Related APIs

`/api/attachments` , `/api/files/:fileId/view|download`
work-planner public-ish attachment view path

Related DB

Attachment

Dependencies

`FILE_MANAGEMENT_API_URL` , `FILE_MANAGEMENT_API_KEY`

Source

`opms-backend/src/modules/attachments|files/` , lead/work-planner attachment modules

Module: Dashboards & Activity

Purpose

Role-scoped KPI summaries and activity feed.

Related APIs

- GET /api/dashboard/admin|sales|finance|dispatch|account|super
- GET /api/activity
- Order stats: GET /api/orders/stats , workflow-context
- Transport planner stats
- Work planner / lead manager dashboard stats on their services

Related UI

Portal overview pages; App/User Manager KPI cards; Lead/Work dashboards

Source

opms-backend/src/modules/dashboard|activity/

API Overview

Style

REST over HTTP/JSON. Multipart for file uploads. SSE for notification streams.

Public entrypoints (defaults)

Service	Base URL (local)	Notes
opms-backend	http://localhost:7001	BFF + order domain; Swagger /api-docs
auth-service	http://localhost:7003	Identity
product-service	http://localhost:7005	Catalog
party-service	http://localhost:7006	Parties/zones/rates
work-planner-backend	http://localhost:7007	/api/work-planner
lead-manager-backend	http://localhost:7009	Leads/quotations
message-service	http://localhost:7011	Messaging
notification-service	http://localhost:7012	Notifications/push

Production hostnames: see nginx [*.spspl.com](#) / [*.medicaent.in](#).

OpenAPI

- **Live:** opms-backend [GET /api-docs](#) generated from `opms-backend/src/docs/swagger.js` + `swaggerExtended.js` (OpenAPI 3.1)
- **Docs copy:** [openapi.yaml](#) — inventory of verified routes across services (may be less detailed than Swagger UI for opms-backend schemas)

Health

All backends: [GET /health](#)

Related

- [Authentication](#)
- [Endpoints](#)

API Authentication

Method

Bearer JWT in Authorization: Authorization: Bearer <token> header.

Obtain token

```
POST /api/auth/login
Content-Type: application/json

{ "email": "user@example.com", "password": "*****" }
```

Hit auth-service (:7003) or via opms-backend proxy (:7001/api/auth/login).

Session introspection

```
GET /api/auth/me
Authorization: Bearer <token>
```

Authorization

Gate	Applies to
requireAuth	Most domain routes
requireOpmsAccess	OPMS domain on opms-backend
requireLeadManagerAccess	Lead Manager mutating/list routes
requireWorkPlannerAccess	Work Planner routes
Webhook secrets	Google Sheet / WhatsApp verify

Portal requirement (OPMS)

User must include portals[] entry { portal_code: "opms", access_roles: [...] }.

Related

- [Architecture Auth](#)

API Conventions

Verified patterns

Pattern	Description
Resource nouns	/api/orders , /api/parties , ...
Soft-delete	DELETE /:id sets deletedAt ; GET /deleted ; POST /:id/restore
Patch updates	Prefer PATCH /:id for partial updates
Nested actions	POST /:id/submit , /approve , /transition
Multipart	Attachments/dispatch bill/due sheet documents via multer field names
Proxies	Same path on BFF as owning service

IDs

MongoDB ObjectId strings in path params.

Pagination / filters

Many list endpoints accept query filters — **exact query schema varies by controller**. Prefer Swagger UI / controller source for each list. Not fully standardized across services.

Idempotency

Not identified as a global idempotency-key convention.

Versioning

No /v1 prefix — **Not identified**.

API Error Handling

Typical HTTP statuses (observed pattern)

Status	Meaning
200/201	Success
400	Validation / bad request
401	Missing/invalid JWT
403	Authenticated but portal/role forbidden
404	Not found
409	Conflict (e.g. unique constraints) — where implemented
500	Server error

Exact error JSON shape varies by service/middleware — **not a single shared error schema across all services**.
Inspect controllers for `{ message }` / `{ error }` patterns.

Workflow transition errors

Invalid next status or blocked by flags returns error from workflow layer (message indicates rule failure).

Source

Express middlewares and module controllers under each service `src/`.

API Endpoints Inventory

Rule: Only endpoints verified from `*.routes.js / app.js` mounts. Request/response bodies: see Swagger (`/api-docs`) for opms-backend where defined; otherwise **Requires confirmation** from controller source.

Authentication column: Public / Bearer / Bearer+Portal / Secret / Internal.

auth-service (:7003)

Method	Endpoint	Auth	Purpose	Source
POST	<code>/api/auth/login</code>	Public	Login	<code>auth.routes.js</code>
GET	<code>/api/auth/me</code>	Bearer	Current user	
POST	<code>/api/auth/change-password</code>	Bearer	Change password	
GET/POST	<code>/api/users</code>	Bearer	List/create users	<code>user.routes.js</code>
GET/PATCH/DELETE	<code>/api/users/:id</code>	Bearer	User CRUD	
GET/POST	<code>/api/users/roles</code>	Bearer	Roles	
POST	<code>/api/users/roles/seed</code>	Bearer	Seed roles	
PUT/DELETE	<code>/api/users/roles/:id</code>	Bearer	Update/delete role	
GET	<code>/api/company-info</code>	Public	Company info	<code>companyInfo.routes.js</code>
GET	<code>/api/company-info/data</code>	Bearer	Data	
PUT/PATCH	<code>/api/company-info</code>	Bearer	Update	
*	<code>/api/portals</code>	Bearer	Portal CRUD + seed	<code>portal.routes.js</code>
*	<code>/api/departments</code>	Bearer	Department CRUD + seed	<code>department.routes.js</code>
GET	<code>/health</code>	Public	Health	

opms-backend domain (:7001) — local routers

Orders `/api/orders` — Bearer + OPMS portal (except webhook)

Method	Path	Notes
POST	<code>/google-sheet-webhook</code>	Public + webhook secret
GET	<code>/</code>	List
GET	<code>/stats</code>	Workflow stats
GET	<code>/workflow-context</code>	Context
POST	<code>/</code>	Create
GET	<code>/deleted</code>	Trash
GET	<code>/:id</code>	Get

Method	Path	Notes
GET	<code>/:id/history</code>	History
GET	<code>/:id/fulfillment</code>	Fulfillment
GET	<code>/:id/approvals</code>	Approvals
GET	<code>/:id/assignees</code>	Assignees
PATCH	<code>/:id</code>	Update
PATCH	<code>/:id/super-sheet</code>	Super sheet patch
DELETE	<code>/:id</code>	Soft-delete
POST	<code>/:id/restore</code>	Restore
POST	<code>/:id/submit</code>	Submit
POST	<code>/:id/transition</code>	Transition status
POST	<code>/:id/close-after-full-delivery</code>	Close helper

Source: `opms-backend/src/modules/orders/order.routes.js`

Order approvals `/api/order-approvals`

Method	Path
GET	<code>/</code> , <code>/deleted</code> , <code>/:id</code>
POST	<code>/</code> , <code>/:id/approve</code> , <code>/:id/reject</code> , <code>/:id/send-to-finance</code> , <code>/:id/send-to-account</code> , <code>/:id/finance-amend</code> , <code>/:id/amend</code> , <code>/:id/resolve-dispatch</code> , <code>/:id/restore</code>
PATCH	<code>/:id</code> , <code>/:id/super-sheet</code>
DELETE	<code>/:id</code>

Finance `/api/finance`

GET `/queue`, GET `/summary`

Approvals `/api/approvals`

GET `/`

Dispatch `/api/dispatch`

List/get/create(with `bill_document`)/patch/soft-delete/restore/deleted

Transport `/api/transport`

Soft-delete CRUD pattern

Order deliveries `/api/order-deliveries`

List/get/patch/soft-delete/restore + POST `/log-shipment`

Order returns `/api/order-returns`

List/get/create/patch/soft-delete/restore

Due sheets `/api/order-due-sheets`

CRUD + document upload + current-by-order + restore

Unbilled `/api/unbilled-orders`

CRUD soft-delete + GET `/order/:orderId`

Final statements `/api/final-order-statements`

GET `/`, GET `/order/:orderId`

Flags `/api/flags`

GET `/`, GET `/:id`, POST `/`, PATCH `/:id`

Dashboard `/api/dashboard`

GET `/admin|/sales|/finance|/dispatch|/account|/super`

Activity `/api/activity`

GET `/`

Attachments `/api/attachments`

List/get/create(multipart)/delete/restore/deleted

Files `/api`

GET `/files/:fileId/view`, GET `/files/:fileId/download`

Fleet

`/api/vehicles`, `/api/drivers`, `/api/transport-agents` — soft-delete CRUD

Reminders `/api/reminders`

Webhook + CRUD + POST `/:id/follow-ups`

Transport planner `/api/transport-plans`

list/stats/eligible-orders/CRUD/submit/complete/cancel/add-remove orders/LR/packed/dispatched/delivered actions

Terms `/api/terms-and-conditions`

No requireAuth on router — GET/POST/PUT/DELETE including text subresources

Proxied (same paths as owning services)

auth, products, parties, notifications, messages/emails/...

product-service (:7005)

Mounts: /api/products , /api/product-groups , /api/product-subgroups , /api/product-brands , /api/product-manufacturers , /api/product-kit-items

Pattern: auth + list/get/create/patch/soft-delete/bulk; products webhook; kit by-kit routes; meta-options on products.

party-service (:7006)

Mount	Highlights
/api/parties	webhook; CRUD; bulk; restore
/api/zones	CRUD; parties; sales-persons
/api/party-products	CRUD; rates approve
/api/party-order-products-rate	check/map helpers

notification-service (:7012)

Method	Path	Auth
POST	/api/notifications/internal/order-transition	Internal (no requireAuth)
POST	/api/notifications/internal/create	Internal
GET	/api/notifications	Bearer
GET	/api/notifications/stream	Bearer SSE
PATCH	/api/notifications/:id/read	Bearer
GET	/api/push/vapid-public-key	Public
POST/DELETE	/api/subscribe	Bearer
POST	/api/push/notify	Bearer

message-service (:7011)

Mount	Endpoints
/api/messages	GET/POST webhook (public); GET/POST send; GET :id (auth)
/api/emails	POST/, GET/, GET/:id
/api/auto-emails	POST /trigger
/api/communication	GET /types; POST /send; /order-received; /queue/:type

lead-manager-backend (:7009)

Mount	Notes
/api/leads	Full CRM actions; list may allow work_planner

Mount	Notes
/api/lead-masters	sources & lost reasons
/api/quotations	admin/manager gates
/api/terms-and-conditions	lead_manager access
/api/attachments	No requireAuth on router
Proxies	notifications, emails, messages, ...

work-planner-backend (:7007)

Base: /api/work-planner

Area	Notes
GET /attachments/:attachmentId/view	Before auth (public view redirect)
plans/visits/works/expenses	CRUD + lifecycle actions
user-settings	Per-user settings
uploads/stats	Supporting

Auth: Bearer + requireWorkPlannerAccess

Example: login

Request

```
POST /api/auth/login HTTP/1.1
Host: localhost:7003
Content-Type: application/json

{
  "email": "admin@example.com",
  "password": "*****"
}
```

Response (shape Inferred — confirm against controller)

```
{
  "token": "<jwt>",
  "user": {
    "email": "admin@example.com",
    "portals": [{ "portal_code": "opms", "access_roles": ["admin"] }]
  }
}
```

Example: transition order

```
POST /api/orders/{id}/transition HTTP/1.1
Host: localhost:7001
Authorization: Bearer <token>
Content-Type: application/json

{
  "to_status": "finance_review",
  "remarks": "optional"
}
```

Exact body field names: confirm in `order.controller` / `service` — do not assume.

Related

- [OpenAPI YAML](#)
- Live Swagger: `/api-docs` on `opms-backend`

Database Overview

Technology

MongoDB via Mongoose. No SQL schemas/migrations framework identified.

Connection

Environment variables: MONGO_URI and/or MONGODB_URI (and sometimes DATABASE_URL fallbacks in code).
Optional MONGODB_LOOKUP_FAMILY=4 for IPv4 preference (Atlas).

Deployment model

MongoDB is external to Docker Compose (typically Atlas). Containers may reach host Mongo via host.docker.internal.

Logical ownership

Domain	Primary service writing schemas
Identity	auth-service models
Orders / logistics / fleet	opms-backend mongoRegistry.js
Products	product-service (mirrors registry)
Parties / rates	party-service (mirrors registry)
Leads / quotations	lead-manager-backend
Work plans	work-planner-backend
Notifications / push	notification-service
Messages	message-service

Inferred: Services share one database/cluster; registries are duplicated. Treat auth + opms-backend + lead + work-planner + notification + message as documentation primaries.

Soft-delete

Shared plugin: deletedAt default filter. See Data Lifecycle.

Source

- opms-backend/src/data/mongoRegistry.js
- auth-service/src/models/*
- service-specific registries

Schema Overview

Schemas are defined in Mongoose (JavaScript), not separate migration SQL.

Canonical order domain

`opms-backend/src/data/mongoRegistry.js` — large multi-model file registering Order and related masters.

Identity

`auth-service/src/models/User.js`, `Role.js`, `Department.js`, `Portal.js`, `CompanyInfo.js`

Warning

`opms-backend/docs/OPMS_SCHEMA.md` and `docs/essentials/*` are **partially stale**. Prefer `mongoRegistry` + workflow modules.

Embedded vs referenced

- Order embeds `order_items[]`
- OrderApproval embeds `approval_items[]`
- Cross-entity links use ObjectId refs (Party, Product, User, ...)

Related

- Collections
- Data Dictionary

Collections Catalog

MongoDB collection names generally follow Mongoose pluralization unless `collection:` is set (e.g. `product_kit_items`, `party_product_last_rates`).

Identity (auth-service)

Model	Purpose
User	Accounts, portals, roles
Role	Role catalog
Department	Department catalog
Portal	Portal catalog + default access_roles
CompanyInfo	Branding & legal/tax profile

Catalog / counterparty

Model	Purpose
Party	Customer/supplier
Zone	Territory grouping
Warehouse	Warehouse master
Customer	Legacy/alternate customer (present in registry)
Product	Product / kit
ProductKitItem	Kit composition
ProductGroup / Subgroup / Brand / Manufacturer	Taxonomy
Batch	Inventory batch
PartyProductMapping / Rate / LastRate	Pricing

Order spine

Model	Purpose
Order	Core order
OrderWorkflow	Append-only workflow actions
OrderStatusHistory	Status changes
OrderApproval	Unified approvals
OrderAmendmentUser	Amendment actors
OrderFlag	Flags
OrderDispatch	Dispatch
TransportShipment	Shipment

Model	Purpose
OrderDelivery	Delivery
OrderReturn	Returns
OrderDueSheet	Due sheets
UnbilledOrder	Unbilled tracking
Reminder	Reminders
Attachment	File metadata
ActivityLog	Activity
TransportPlan / TransportPlanOrder	Transport planner
TransportAgent / Vehicle / Driver	Fleet
TermsAndConditions / TermsText	T&C

CRM

Model	Purpose
Lead	Lead
LeadFollowUp	Follow-ups
LeadSource / LeadLostReason	Masters
LeadQuotation	Quotations
FollowUpDigestLog	Digest dedupe

Work planner

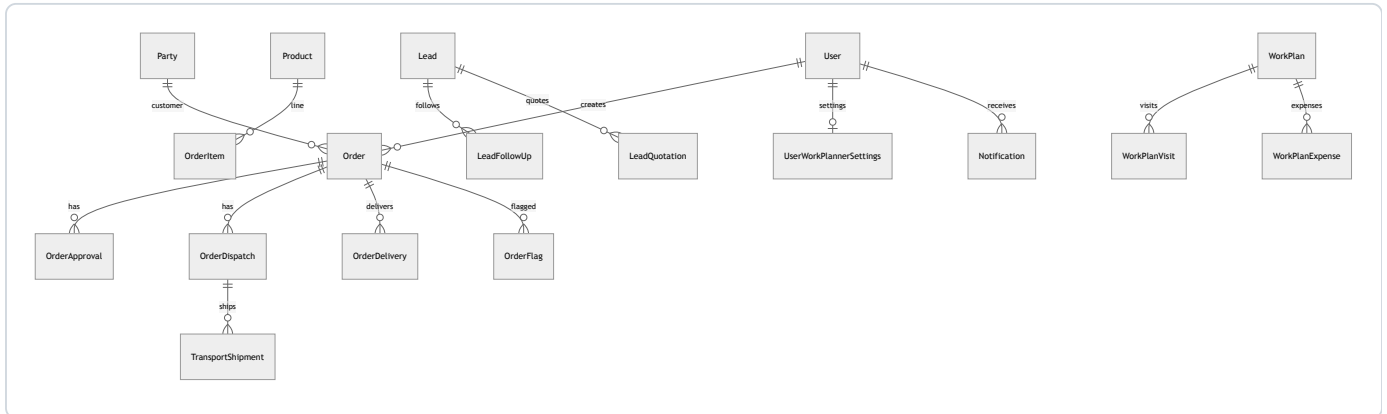
Model	Purpose
WorkPlan	Daily plan
WorkPlanVisit / WorkPlanWork / WorkPlanExpense	Children
UserWorkPlannerSettings	Per-user settings

Comms

Model	Purpose
Notification	In-app notification
PushSubscription	Web push
Message	Outbound message log

Relationships

Logical relationships (Mongo does not enforce FKs):



Notable refs

- Order → party, customer, lead, created_by
- OrderApproval → order
- PartyProductRate → party, product, mapping
- Attachment → entity_type + entity_id (polymorphic)
- TransportShipment → transporter ref **without registered Transporter model** (debt)

Related

- ERD

Indexes

Important indexes observed in schemas (non-exhaustive):

Model	Index notes
User	email unique; company_id
Portal	code unique
Role	code unique
Order	order_no unique; compound indexes including deletedAt
Party	party_name ; sparse gst_no ; featured+active
Product	sku unique sparse; text/ searchable_text
ProductSubgroup	unique (name, group)
Batch	unique (product, batch_no)
PartyProductMapping	unique (party, product)
PartyProductLastRate	unique (party, product)
WorkPlan	unique (sales_user, plan_date) where deletedAt null
WorkPlanVisit	unique (work_plan, sequence)
Lead	lead_no unique
PushSubscription	endpoint unique
Message	indexes on order/recipient/channel/status
FollowUpDigestLog	unique kind+user+digest_date trio
CompanyInfo	is_default

Caveat: OrderDispatch index referencing status while field is dispatch_status may be incorrect — **Requires confirmation.**

Source

Schema index: true / schema.index(...) in mongoRegistry and models.

Constraints

Uniqueness

See [Indexes](#) for unique indexes.

Enums (selected)

Order.status

`draft`, `submitted`, `sales_approved`, `finance_review`, `finance_approved`, `finance_rejected`, `account_review`, `account_approved`, `account_rejected`, `dispatch`, `in_transit`, `delivered`, `closed`, `cancelled`, `on_hold`

Note: `closed` in schema but absent from transition graph.

Lead.status

`new`, `assigned`, `contacted`, `qualified`, `unqualified`, `follow_up`, `quotation`, `negotiation`, `won`, `lost`, `converted`

WorkPlan.status

`planned`, `draft`, `submitted`, `approved`, `rejected`, `completed`

Required fields

Vary by model — see Data Dictionary / schema `required: true`.

Application-level constraints

Workflow transition graph; flag blocking; portal RBAC — not DB-enforced.

Data Lifecycle

Soft-delete

- Field: `deletedAt` (Date or null)
- Plugin filters all queries to active docs unless `.withDeleted()`
- Restore clears `deletedAt`

Append-only

- `OrderWorkflow`, `OrderStatusHistory`, `ActivityLog` — no soft-delete in typical config

Message lifecycle

`pending` → `queued` → `sending` → `sent|failed` with attempts metadata

Notification lifecycle

Created unread → marked read (`is_read` , `read_at`)

Due sheets

`active` → `superseded` / `archived`; `is_current` flag

Backup

Not identified automated Mongo backup in repo. Ops must use Atlas/host tooling.

Data Dictionary (selected)

Descriptions only where purpose is clear from field names/code. Unknowns marked.

Collection/Model	Field	Type	Required	Default	Description	Relationship
User	email	String	yes	—	Login identity	unique
User	password	String	yes	—	Hash (select:false)	—
User	portals	Array	no	[]	Portal assignments	→ Portal
User	portals.portal_code	String	—	—	Portal code denorm	—
User	portals.access_roles	[String]	—	—	Roles in portal	—
User	department	String	no	—	Department code/name	—
User	is_active	Boolean	—	—	Active flag	—
Portal	code	String	yes	—	Unique portal code	unique
Portal	access_roles	[String]	—	—	Default roles list	—
Order	order_no	String	—	—	Business number	unique
Order	status	String enum	—	—	Workflow status	—
Order	lifecycle_status	String enum	—	—	Lifecycle summary	—
Order	workflow_stage	String enum	—	—	Stage bucket	—
Order	party	ObjectId	—	—	Counterparty	→ Party
Order	order_items	Array	—	—	Line items	embedded
Order	deletedAt	Date	—	null	Soft-delete	—
OrderApproval	order	ObjectId	yes	—	Parent order	→ Order
OrderApproval	is_admin_approved	Boolean	—	false	Admin signature	—
OrderApproval	is_finance_approved	Boolean	—	false	Finance signature	—
OrderApproval	is_account_approved	Boolean	—	false	Account signature	—
Party	party_type	enum	—	—	customer/supplier/both	—
Party	party_name	String	—	—	Display name	—
Product	product_type	enum	—	—	individual/kit	—
Product	sku	String	—	—	SKU	unique sparse
Lead	status	enum	—	—	Pipeline status	—
Lead	assigned_*	mixed	—	—	Assignment fields	→ User
WorkPlan	plan_date	Date	—	—	Plan day	unique w/ user
WorkPlan	status	enum	—	—	Plan status	—
Notification	is_read	Boolean	—	—	Read state	—
Message	channel	enum	—	—	email/whatsapp	—
Message	status	enum	—	—	Delivery status	—

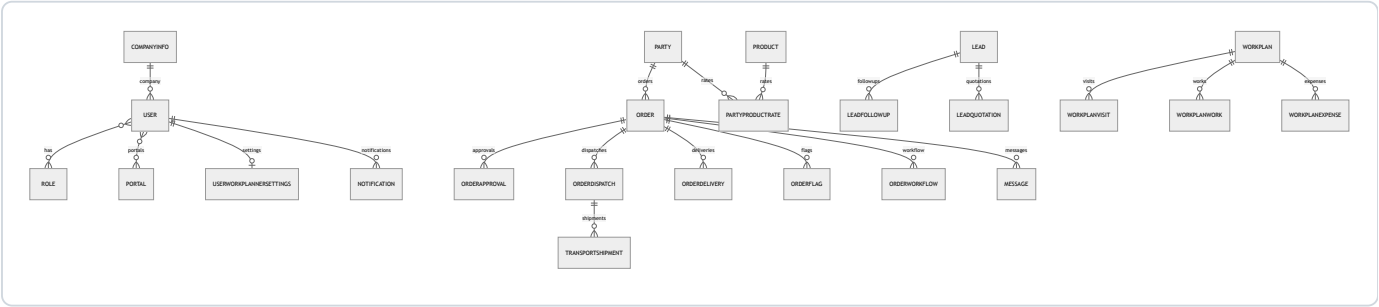
Collection/Model	Field	Type	Required	Default	Description	Relationship
Attachment	entity_type	enum	—	—	Owner type	polymorphic
Attachment	entity_id	ObjectId/String	—	—	Owner id	—

Full field lists: see `mongoRegistry.js` and service models — too large to duplicate verbatim; regenerate from source for audits.

Related

- Collections

ERD



Screenshot: Not available in repository.

Development Overview

OPMS is a polyglot-free **Node.js + Next.js monorepo** orchestrated with Docker Compose.

Typical developer loop

1. Configure `.env.docker` from example (use placeholders for secrets)
2. Start stack with compose (optionally `docker-compose.dev.yml` for hot reload)
3. Open App Frontend `:7013` or OPMS `:7002`
4. Use Swagger at `:7001/api-docs` for OPMS API exploration

Packages

13 `package.json` projects (8 backends + 5 frontends). No root workspace package manager identified.

Related

- Local Development
- DOCKER.md

Prerequisites

Tool	Purpose	Evidence
Docker Desktop / Engine + Compose v2	Run full stack	DOCKER.md
Node.js 20	Image base <code>node:20-alpine</code> ; local non-docker optional	Dockerfiles
Git	Source control	<code>.git</code>
MongoDB Atlas or local Mongo	Database	MONGO_URI
Optional: host nginx + certbot	TLS prod	nginx/

Not required for basic Docker run

Kubernetes, RabbitMQ, local MinIO (file API external).

Local Development

Steps (verified from DOCKER.md)

1. Clone repository
2. `cp .env.docker.example .env.docker`
3. Edit `.env.docker`: set `JWT_SECRET`, `MONGO_URI` / `MONGODB_URI` (**do not commit secrets**)
4. `ln -sf .env.docker .env`
5. Start:
 - Prod-style: `docker compose --env-file .env.docker up --build`
 - Dev hot reload: `docker compose --env-file .env.docker -f docker-compose.yml -f docker-compose.dev.yml up --build`
6. Verify: `http://localhost:7001/health` and App Frontend `http://localhost:7013`
7. Optional seed: `docker compose --env-file .env.docker exec opms-backend npm run seed:users`

Per-service local run (without compose)

Possible via each package `npm run dev / start` if env vars and Mongo/Redis are provided — exact standalone runbooks **partially documented**; prefer Compose.

URLs

See DOCKER.md table (7001–7014).

Project Structure

```
/
├─ app-frontend/           # SSO hub Next.js
├─ opms-frontend/         # OPMS UI
├─ user-manager-frontend/
├─ lead-manager-frontend/
├─ work-planner-frontend/
├─ auth-service/
├─ opms-backend/           # BFF + order domain
├─ product-service/
├─ party-service/
├─ lead-manager-backend/
├─ work-planner-backend/
├─ message-service/
├─ notification-service/
├─ nginx/                 # Host TLS reverse proxy configs
├─ docker-compose.yml
├─ docker-compose.dev.yml
├─ docker-compose.prod.yml # Partial stack
├─ DOCKER.md
├─ .env.docker.example
├─ docs/                  # This documentation suite
├─ backups/               # gitignored leftovers
└─ minio-data/            # leftover, not compose service
```

Typical backend layout

```
src/server.js , src/app.js , src/modules/ , src/models OR src/data/mongoRegistry.js , src/middlewares/ ,
src/config/
```

Environment Variables

IMPORTANT: Never copy real secrets into documentation.

SECRET DETECTED IN SOURCE — NOT INCLUDED IN DOCUMENTATION: `.env.docker.example` currently contains values that look like live JWT secrets, Mongo credentials, API keys, and passwords. Treat as a credential-hygiene incident; rotate if exposed; replace example file with placeholders.

Variable	Required	Purpose	Example (placeholder)	Secret
NODE_ENV	yes	Runtime mode	production	no
JWT_SECRET	yes	JWT signing	change-me-long-random	yes
JWT_EXPIRES_IN	no	Token TTL	7d	no
MASTER_PASSWORD	no	Break-glass login	empty in prod	yes
JSON_BODY_LIMIT	no	Body size	50mb	no
MONGO_URI / MONGODB_URI	yes	Mongo connection	mongodb+srv://USER:PASS@host/db	yes
MONGODB_LOOKUP_FAMILY	no	Prefer IPv4	4	no
REDIS_URL	no	Redis	redis://redis:6379	no*
REDIS_PORT	no	Host publish	7014	no
*_PORT	no	Host ports	see compose defaults	no
AUTH_SERVICE_URL etc.	no	Internal URLs	http://auth-service:7003	no
API_PUBLIC_BASE_URL	prod	Public API	https://api-opms.example.com	no
NEXT_PUBLIC_*	prod	Browser URLs (build-time)	https://...	no
CORS_ORIGINS	prod	Extra CORS	comma URLs	no
COMPANY_NAME / LOGO	no	Branding fallback	—	no
SEED_*	no	Seed controls	—	SEED_PASSWORD yes
FOLLOWUP_REMINDER_*	no	Lead cron	—	no
FILE_MANAGEMENT_API_URL/KEY	optional	Uploads	—	key yes
SMTP_*	optional	Email	—	pass yes
MICROSOFT_GRAPH_*	optional	Email	—	secret yes
WHATSAPP_*	optional	WhatsApp	—	yes

Variable	Required	Purpose	Example (placeholder)	Secret
VAPID_*	optional	Web push	—	private yes
GOOGLE_SHEET_WEBHOOK_SECRET	optional	Webhooks	—	yes
ADMIN_EMAIL / ACCOUNT_EMAIL / FINANCE_EMAIL / DUE_SHEET_EMAIL / DISPATCH_EMAIL	optional	Role mailboxes	—	no
COMPOSE_PROJECT_NAME / ENV_FILE	no	Multi-stack	—	no
APP_LOGIN_URL / LEAD_MANAGER_PUBLIC_URL / FRONTEND_URL	optional	Links in emails	—	no

* Redis currently unauthenticated in compose.

Source: `.env.docker.example` , service `env.js` files, compose.

Configuration

Patterns

- `dotenv / compose env_file`
- Service `src/config/env.js` reading `process.env`
- Frontend `NEXT_PUBLIC_*` baked at **image build**

Do not

Re-declare `MONGO_URI / JWT_SECRET` under `compose environment:` with empty interpolation — overrides `env_file` (DOCKER.md).

Nginx

Copy snippets + site files to `/etc/nginx/` ; use HTTP bootstrap before certbot; then HTTPS site.

Running the Application

Full stack

```
docker compose --env-file .env.docker up --build
# or detached
docker compose --env-file .env.docker up -d --build
```

Dev hot reload

```
docker compose --env-file .env.docker -f docker-compose.yml -f docker-compose.dev.yml up --build
```

Partial prod file (incomplete)

```
docker compose --env-file .env.docker -f docker-compose.prod.yml up --build -d
```

Warning: missing many services vs full product.

Useful

```
docker compose --env-file .env.docker logs -f opms-backend
docker compose --env-file .env.docker down
docker compose --env-file .env.docker down -v # wipes Redis volume
docker compose --env-file .env.docker exec opms-backend npm run seed:users
```

Package scripts (examples)

- Backends: `npm start`, `npm run dev`
- Frontends: `npm run dev`, `npm run build`, `npm start`, `npm run lint`
- opms-backend also has migrate/seed scripts in `package.json`

Testing

Status

Not identified as a comprehensive automated test suite at monorepo root (no Jest/Playwright configs systematically documented across all packages in discovery).

What exists

- Lint scripts on frontends (`npm run lint`)
- Health endpoints for smoke checks
- Manual verification via UI + Swagger

Recommendation

Requires confirmation from engineering on intended test strategy; do not invent coverage claims.

Code Quality

Mechanism	Where
ESLint via <code>next lint</code>	Frontends package scripts
Swagger for API surface	opms-backend <code>/api-docs</code>
Soft-delete plugin consistency	Shared plugin copies

No monorepo-wide prettier/eslint root config identified as mandatory CI gate (no CI).

Git Workflow

Observed

- Git repository at monorepo root
- `.gitignore` excludes `.env.docker` , backups, minio-data, etc.

Formal branching / PR policy

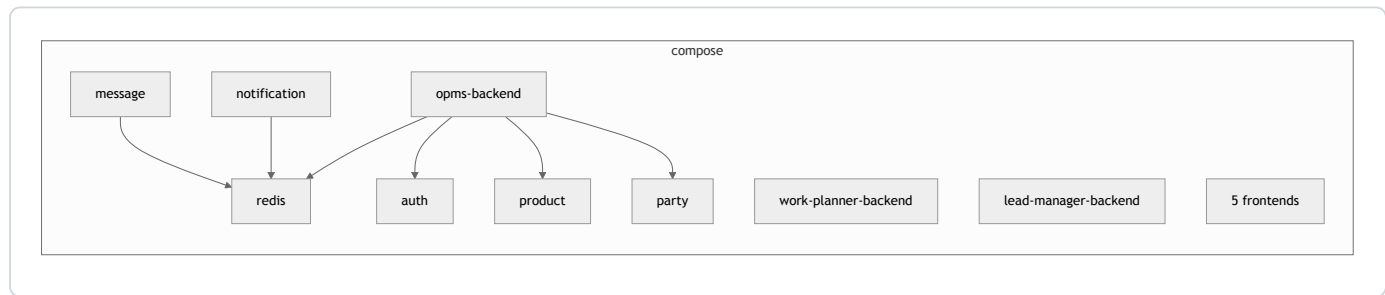
Not identified in repository (no CONTRIBUTING.md).

Requires confirmation from team process.

Docker

Compose services (base)

redis + 8 APIs + 5 frontends. See [Infrastructure Architecture](#).



Images

Build contexts per service folder; `node:20-alpine` .

Volumes

`redis_data` only.

Healthchecks

Backends + redis; frontends none in base compose.

Dev overlay

`Dockerfile.dev` , bind-mount `src` , watch modes, FE healthchecks disabled.

Related

Root [DOCKER.md](#)

Deployment

Path A — Single host Docker + nginx (documented)

1. Provision host with Docker + nginx + certbot
2. Configure `.env.docker` with production URLs and secrets
3. `docker compose --env-file .env.docker up -d --build`
4. Install nginx site (`nginx/sites-available/opms`) and obtain certs (commands in nginx file headers)
5. DNS for all hostnames
6. Verify `/health` on APIs and FE loads

Path B — Medica tenant

Use `.env.docker.medica` + medica nginx; ports 5001–5013. Dual-stack on one host **Requires confirmation**.

Image registry / K8s

Not identified.

Rollback

Rebuild/redeploy previous git SHA; no automated rollback tool in repo.

Production Runbook

Startup

```
docker compose --env-file .env.docker up -d
```

Health verification

```
curl -fsS http://127.0.0.1:7001/health
curl -fsS http://127.0.0.1:7003/health
# repeat for other API ports
```

Logs

```
docker compose --env-file .env.docker logs -f <service>
```

Restart one service

```
docker compose --env-file .env.docker restart opms-backend
```

Seed / admin bootstrap

Seed scripts + User Manager to assign portals. Exact first-admin procedure **Requires confirmation**.

Incidents

See Troubleshooting.

Monitoring

Capability	Status
/health endpoints	Implemented on backends
Redis healthcheck	Compose <code>redis-cli ping</code>
Prometheus/Grafana/APM	Not identified
Uptime checks	Not identified in repo
Alerting	Not identified

SSE streams exist for notifications — not a metrics system.

Logging

- Application logs via container stdout/stderr
- Access via `docker compose logs`
- No ELK/Loki stack in compose

Structured logging standard: **Not identified** as global.

Backup and Restore

What needs backup

Data	Location	Repo automation
MongoDB	Atlas/external	Not identified
Redis	redis_data volume	Not identified (often ephemeral queues)
Files	External File API / MinIO	Not identified
Config	.env.docker (secrets store outside git)	operator-managed
nginx certs	host /etc/letsencrypt	operator-managed

Observed artifacts

backups/minio-backup-*.tar.gz leftover; dump.rdb gitignored — **not** a documented procedure.

Commands

Not identified official backup/restore scripts. Use MongoDB Atlas backups / mongodump per ops standards — **Requires confirmation.**

Disaster Recovery

Status

No formal DR runbook in repository.

Inferred recovery outline (not official)

1. Restore Mongo from Atlas PITR/snapshot
2. Redeploy compose stack from known-good git tag
3. Restore env secrets from secure vault
4. Re-point DNS/nginx if host replaced
5. Verify login + create-order smoke test

Requires confirmation from ops for RTO/RPO targets.

Troubleshooting

Compose won't start / env overrides

Symptoms: Mongo auth failures despite `env_file`

Cause: Empty `${MONGO_URI}` in `environment:` overrides file

Solution: Follow DOCKER.md — don't redeclare secrets in environment

Verify: `docker compose exec auth-service printenv MONGO_URI` (ensure set; don't log in shared chats)

JWT errors across services

Symptoms: 401 after login when calling another service

Cause: Mismatched `JWT_SECRET`

Solution: Single shared secret in `.env.docker`

Verify: Relogin; call `/api/auth/me`

403 on OPMS routes

Symptoms: Authenticated but forbidden

Cause: Missing `portals[]` entry `portal_code: opms`

Solution: Assign portal in User Manager; re-login

Verify: `/api/auth/me` shows portals

Frontend calls wrong API / CORS

Symptoms: Network errors to localhost from prod build

Cause: `NEXT_PUBLIC_*` baked at build

Solution: Fix env and **rebuild** frontend images

Verify: View page source / network hostnames

Redis / queue jobs stuck

Symptoms: Notifications/emails not sending

Cause: Redis down or workers not started

Solution: Check redis health; restart message/notification/opms-backend

Verify: `docker compose ps` ; service logs

Mongo Atlas IPv6 issues

Symptoms: Connection timeouts from containers

Cause: DNS family

Solution: `MONGODB_LOOKUP_FAMILY=4`

Verify: Backend connects and `/health` OK

File upload failures

Symptoms: Attachment create errors

Cause: Missing `FILE_MANAGEMENT_API_*`

Solution: Configure URL/key or disable upload features

Verify: Upload small file in UI

nginx 502

Symptoms: Public hostname 502

Cause: Container not listening / wrong upstream port

Solution: Align nginx upstream with compose ports; `nginx -t` + reload

Verify: curl upstream on 127.0.0.1:port

Security

Implemented

- JWT auth + bcrypt passwords
- Portal RBAC
- HTTPS via nginx TLS configs
- Webhook secrets for Google Sheets
- Soft-delete vs hard-delete for many records

Risks identified in codebase (document, do not "fix" here)

Risk	Notes
Secrets in <code>.env.docker.example</code>	Appears to contain real credentials — rotate & sanitize
<code>MASTER_PASSWORD</code>	Break-glass; must be empty in prod
Redis without AUTH + published port	Network exposure risk
Notification internal routes unauthenticated	Network trust assumed
opms terms-and-conditions router without <code>requireAuth</code>	Exposure risk
lead-manager attachments router without <code>requireAuth</code>	Exposure risk
work-planner attachment view before auth	Intentional public view? Requires confirmation
Duplicated schema registries	Drift / authz inconsistency risk

Rate limiting

Not identified.

Related

- Auth architecture

Performance

Config levers found

- `JSON_BODY_LIMIT`
- nginx proxy timeouts / body size for API hosts
- Redis/BullMQ async offload for messages/notifications/some order jobs
- Mongo indexes on hot fields

Targets

No published latency/throughput SLOs in repo.

Frontend

Next.js production builds; `NEXT_PUBLIC_*` must be correct to avoid extra failures (not perf per se).

User Guide

Welcome to the OPMS platform applications. This guide is based on the screens and routes present in the frontends.

Screenshot: Not available in repository.

Applications you may use

App	What it is for
App (login hub)	Sign in and open your assigned portals
OPMS	Orders, parties, products, dispatch, transport planning
User Manager	Users, portals, departments, company branding (admins)
Lead Manager	Leads, follow-ups, quotations
Work Planner	Daily plans, visits, expenses

How access works

Your administrator assigns **portals** and **roles**. If you cannot open an app, you likely lack that portal assignment.

Related

- [Getting Started](#)
- [Role Guides](#)

Getting Started

1. Open the App login URL (local default `http://localhost:7013` , production e.g. `https://app.spspl.com`).
2. Sign in with the email and password provided by your administrator.
3. On your department dashboard, open an **Assigned Portal** tile.
4. The portal opens with single sign-on (`?token=` is applied automatically).
5. Use the sidebar to navigate features for your role.

First-time tips

- Change your password from Profile if available.
- Ensure pop-up/redirects are allowed if an admin opens User Manager in an embed.
- If branding looks wrong, your admin should update Company Information in User Manager.

Screenshot: Not available in repository.

Login

App Frontend

1. Go to `/`
2. Enter email and password
3. Submit
4. On success you reach `/dashboard/...`

Portal-specific login pages

OPMS, Lead Manager, Work Planner, and User Manager also expose `/` or `/login` if opened directly. Prefer the App hub when possible for SSO.

SSO

When launched from App, a token is passed in the URL once and stored in the browser session storage/cookies used by that app.

Logout

Use the user menu / logout control in the top bar (exact label may vary by app).

Problems

See [Troubleshooting](#).

Navigation

App Frontend

- Sidebar: roles under your department
- Main: Assigned Portals grid
- Topbar: theme, notifications, profile

OPMS

Portal path prefix: `/admin` , `/sales` , `/finance` , `/dispatch` , `/account` , `/super_admin`

Typical sidebar groups (by role):

- Overview
- Order Master / Create Order
- Party Master / Product Master (not sales/dispatch limited views)
- Transport Agent Master / Transport Planner (dispatch-focused roles)

Lead Manager

Dashboard · Leads · Follow-Ups · Reports · Quotations* · Terms*

* admin/manager

Work Planner

Dashboard · Work Plans · Tasks & Visits · Calendar · Expenses · Team Manager/My Team*

* admin/manager

User Manager

Dashboard · User Directory · Portals · Departments & Roles · Company Info

Screenshot: Not available in repository.

Feature Guides

Creating an OPMS order

1. Login and open OPMS portal for your role.
2. Go to **Create Order**.
3. Select party and add products/quantities/rates as the form requires.
4. Save draft.
5. **Submit** when ready for workflow.
6. Track status in **Order Master** / order detail tabs (approvals, dispatch, etc.).

Approving an order (finance/account/admin)

1. Open order detail or approvals queue.
2. Review line items and due sheet if present.
3. Use Approve / Reject / Send to finance or account actions available to your role.
4. Confirm status advanced in order header.

Transport planning

1. Open **Transport Planner**.
2. Create a plan for a date/agent.
3. Add eligible orders.
4. Submit and update packed/dispatched/delivered as operations proceed.

Managing a lead

1. Open Lead Manager → Leads → New.
2. Enter contact and requirement details.
3. Assign and schedule follow-ups.
4. Optionally create a quotation (manager/admin).
5. Mark won/lost or convert when appropriate.

Creating a work plan

1. Open Work Planner → Work Plans → New.
2. Choose plan type and date.
3. Add visits/works.
4. Submit for approval (if required by your process).
5. Check in/complete visits; submit expenses.

Admin: assign portal access

1. Open User Manager → User Directory → user detail / portals.
2. Assign portal codes and access roles.
3. Ask the user to re-login via App hub.

Screenshot: Not available in repository.

Role Guides

OPMS — Sales

Purpose: Create and track sales orders.

Features: Overview, Order Master, Create Order.

Restrictions: Limited masters/nav vs admin; own-order emphasis on overview.

OPMS — Finance

Purpose: Commercial/financial review.

Features: Orders, approvals/finance queues, masters, transport planner (per nav).

Typical tasks: Review approvals, due sheets, risk flags.

OPMS — Account

Purpose: Account-stage review before dispatch.

Features: Similar masters to finance/admin per nav.

Typical tasks: Account approve/reject/amend.

OPMS — Dispatch

Purpose: Fulfillment and logistics.

Features: Orders, transport agents, transport planner; drivers/vehicles routes exist.

Typical tasks: Create dispatches, run transport plans, log deliveries.

OPMS — Admin / Super Admin

Purpose: Broad operational control; super admin also User Manager embed.

Features: Full masters + orders + transport; users via User Manager.

Lead Manager — Executive

Purpose: Personal pipeline.

Features: Leads, follow-ups, my reports.

Restrictions: No quotations/terms nav.

Lead Manager — Manager / Admin

Purpose: Team CRM + quotations.

Features: Full leads, quotations, terms, sales reports (admin labeling).

Work Planner — Executive

Purpose: Own daily plan and expenses.

Restrictions: No team manager sidebar.

Work Planner — Manager / Admin

Purpose: Approve plans/expenses; configure assignees.

Features: Team Manager / My Team + settings.

User Manager — Admin

Purpose: Identity & branding administration.

Features: Users, portals, departments/roles, company info.

Common Tasks

Task	Steps (summary)
Launch a portal	App dashboard → Assigned Portals tile
Create order	OPMS → Create Order → save → submit
Find order	OPMS → Order Master → search/filters
Raise flag	Order detail → Flags → create
Add reminder	Reminders UI / order context
Upload attachment	Order/lead/expense attachment controls
Import via Google Sheet	Use sync/import UI where available (admin/configured)
Change password	Profile → Password change panel
Approve work plan	Work Planner → plan → Approve
Submit expense	Work Planner → Expenses → submit
Create quotation	Lead Manager → Quotations → New (manager/admin)
Assign user portal	User Manager → user/portals

Screenshot: Not available in repository.

User Troubleshooting

I can log in but portals are empty

Ask an admin to assign portals in User Manager.

OPMS says I don't have access

You need the `opms` portal with an access role (sales/finance/...). Re-login after assignment.

I opened a bookmark and must log in again

Sessions are browser-stored; use App hub launch for SSO. Clear stale tabs with old tokens.

Notifications not appearing

Check notification bell permissions; ensure you are logged in; ask ops if notification-service is down.

Upload failed

File service may be unavailable — contact administrator.

Wrong company logo/name

Admin updates Company Information; you may need a refresh.

FAQ

Q: What is OPMS?

A: The operations portal for orders, masters, and logistics, part of a multi-app platform.

Q: Why are there multiple websites?

A: Each domain (orders, leads, plans, users, login hub) is a separate app for clarity and access control.

Q: Can one person have multiple roles?

A: Yes — users can hold multiple portal access_roles and department roles as assigned.

Q: Where do I manage passwords for other users?

A: User Manager (admin). End users change their own password in Profile.

Q: Is there a mobile app?

A: Not identified in this repository; use the responsive web apps.

Q: Who do I contact for access?

A: Your system administrator / User Manager operator. Support escalation path: **Requires confirmation.**